> 本笔记基于 MIT OCW 18.404J《Theory of Computation》(Fall 2020) 的公开材料整理：25 份讲义 PDF（L1–L12、L14–L26，L13 为期中考试无讲义）、25 份视频逐字稿、6 份 problem set、3 份 sample exam 与 1 份解答，以及 syllabus / calendar / readings / assignments / exams 页面。教材为 Sipser《Introduction to the Theory of Computation》第 3 版（Cengage, 2012, ISBN 9781133187790）。所有讲次编号、章节号、评分比例、定理名均可在文末「参考来源」列出的原始 PDF 与页面中逐条核对；凡属我个人的类比、教学经验或课程未覆盖的补充内容，均显式标注「推断」「课外」或「工程补充」。材料访问日期统一为 **2026-11-02**。

0.1 这门课在教什么
0.2 课程基本事实（可直接核对）
0.3 先修要求（Syllabus 原文口径）
0.4 评分构成
0.5 26 讲地图
0.6 材料形态与抓取口径
0.7 与 CS170 的分工：为什么两门课都要学
0.8 建议的读法
1.1 问题：一个只会"记状态"的机器能认多少语言
1.2 形式定义
1.3 定理与证明思路
1.4 例子
1.5 常见误区
2.1 问题：并集已经闭包了，串接卡在哪
2.2 形式定义
2.3 定理与证明思路
2.4 例子
2.5 常见误区
3.1 问题：否证比构造难一个量级
3.2 形式定义
3.3 定理与证明思路
3.4 例子
3.5 常见误区
4.1 问题：正则表达式写不出嵌套
4.2 形式定义
4.3 定理与证明思路
4.4 例子
4.5 常见误区
5.1 问题：栈到底买到了什么
5.2 形式定义
5.3 定理与证明思路
5.4 例子
5.5 常见误区
6.1 问题：为什么正则那套泵法不够用
6.2 形式定义
6.3 定理与证明思路
6.4 例子
6.5 常见误区
7.1 问题：自动机加了什么才够"通用"
7.2 形式定义
7.3 定理与证明思路
7.4 例子
7.5 常见误区
8.1 问题：凭什么"这一个模型"代表所有计算
8.2 形式定义
8.3 定理与证明思路
8.4 例子
8.5 常见误区
9.1 问题：把"问题"变成"语言"
9.2 形式定义
9.3 定理与证明思路
9.4 例子
9.5 常见误区
10.1 问题：怎么"证明不存在"
10.2 形式定义
10.3 定理与证明思路
10.4 例子
10.5 常见误区
11.1 问题：只有一个不可判定问题，怎么长出更多
11.2 形式定义

11.3 定理与证明思路
11.4 例子
11.5 常见误区
12.1 问题：当待证问题"看起来跟 TM 无关"
12.2 形式定义
12.3 定理与证明思路
12.4 例子
12.5 常见误区
13.1 问题：自指看起来是非法的
13.2 形式定义
13.3 定理与证明思路
13.4 例子
13.5 常见误区
14.1 问题：复杂度理论的第一道坎是"模型依赖"
14.2 形式定义
14.3 定理与证明思路
14.4 例子
14.5 常见误区
15.1 问题：非确定性怎么变成"可验证"
15.2 形式定义
15.3 定理与证明思路
15.4 例子
15.5 常见误区
16.1 问题：怎么知道"这个 NP 问题是最难的"
16.2 形式定义
16.3 定理与证明思路
16.4 例子
16.5 常见误区
17.1 问题：空间比时间"便宜"在哪
17.2 形式定义
17.3 定理与证明思路
17.4 例子
17.5 常见误区
18.1 问题：亚线性空间怎么定义
18.2 形式定义
18.3 定理与证明思路
18.4 例子
18.5 常见误区
19.1 问题：类之间是不是"真的"不同
19.2 形式定义
19.3 定理与证明思路
19.4 例子
19.5 常见误区
20.1 问题：允许"掷硬币"会不会变强
20.2 形式定义
20.3 定理与证明思路
20.4 例子
20.5 常见误区
21.1 问题：证明一定要"写下来"吗
21.2 形式定义
21.3 定理与证明思路
21.4 例子
21.5 常见误区
22.1 给一个语言，按顺序问六个问题
22.2 三个必须记住的"为什么"
22.3 一个课外补丁
23.1 三个判据
23.2 CNF 转换的四件事
23.3 一条不能推广的推论
24.1 三个模板
24.2 方向：唯一的失分大项
24.3 为什么课程禁用 Rice 定理
24.4 硬度阶梯
25.1 已知与未知的分界
25.2 完全性的归约必须比类更弱
25.3 电路复杂性概览（课外）
25.4 一句话总结
26.1 ← CS70：证明的地基
26.2 ← CS170：归约的工程版
26.3 ← 6.006 与 18.405
27.1 复习清单（按"必须能当场写出"分级）
27.2 自测题（10 题，答案紧随）
答案
28.1 课程页面
28.2 讲义 PDF（25 份）
28.3 作业与考试
28.4 教材
28.5 声明

第 0 章 课程概览与讲次地图

0.1 这门课在教什么

18.404J 是 MIT 的**计算理论**主课，与 18.4041J、6.840J 是三号同一门课（讲义页脚写的是 "18.404J / 18.4041J / 6.840J Theory of Computation"）。课程描述（Syllabus 页原文）把内容分成两块：**可计算性（computability）**与**计算复杂性（computational complexity）**。第一讲讲义开篇给了一张时间轴，把两块的分工讲得很清楚：

板块	年代	核心问题	典型问题
可计算性理论	1930s–1950s	什么是可计算的，什么不是	程序验证、数学真理
复杂性理论	1960s–至今	什么在实践中可计算	因数分解、P vs NP、时间与空间度量

这两块共享同一套方法：**先形式化"计算"这个动作，再对形式化对象证明定理**。这也是本课与算法课最本质的差别——算法课问"怎么算得快"，本课问"算"这个字到底指什么，以及它的边界在哪。讲义的措辞是 "Models of Computation: Finite automata, Turing machines, …"。

一个容易被忽略的定位细节：Sipser 在 L1 把"理论在计算机科学中的角色"列为四条——应用、基础研究、与其他领域的联系、计算的本质。这条列表解释了为什么一门纯数学味道的课会出现在 6 系（EECS）的培养方案里。

0.2 课程基本事实（可直接核对）

项目	内容	来源
课程编号	18.404J / 18.4041J / 6.840J	讲义页脚
学期	Fall 2020（全程线上，因 Covid-19 校园关闭）	Syllabus 页
授课教师	Prof. Michael Sipser	OCW 课程信息栏、讲义封面
助教（L1 列出）	Fadi Atieh、Damian Barabonkov、Alex Dimitrakakis、Thomas Xiong、Abbas Zeitoun、Emily Liu	讲义 L1 第 1 页
上课形式	讲座每周 2 次 × 1.5 小时；复习课（recitation）每周 1 次 × 1 小时	Syllabus 页
教材	Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage, 2012, ISBN 9781133187790	Readings 页
材料形态	25 份讲义 PDF + 25 份视频逐字稿 + 6 份 problem set + 3 份 sample exam + 1 份 final 解答	课程 zip 包
讲次总数	26 讲 + 期中（L13）+ 期末（L27 位置）	Calendar 页

助教名单只出现在 L1 讲义首页，OCW 页面上没有单独列出——这是"讲义 PDF 比网页信息更细"的一个典型例子，也是我坚持逐份下载 PDF 而不是只读网页的原因。

0.3 先修要求（Syllabus 原文口径）

先修课	大纲写的是什么
6.042J Mathematics for Computer Science	离散数学、逻辑与证明、图论、概率
18.200 Principles of Discrete Applied Mathematics	同上，数学系版本

L1 讲义对先修的描述更直白，值得抄下来：**"Prior substantial experience and comfort with mathematical concepts, theorems, and proofs. Creativity will be needed for psets and exams."** 这句话是本课最真实的难度说明——它不是"读完就能懂"的课，而是"必须动手写证明"的课。作业协作政策也写在这一页：允许讨论，但必须自己写解答，且**不允许查"bible"（历年题解汇编）或在线材料**。

0.4 评分构成

项目	占比	说明
Homework	35%	6 份双周作业；L2 讲义补充：迟交按每个问题扣 1 分（满分 10 分），次日 23:59 后不再接受（除非有 S3 病假）
Quizzes	25%	每次直播讲座中的随堂 quiz；录播观看者有对应的限时版本，48 小时内完成即计分
Midterm Exam	15%	课内进行（即 L13）；**90 分钟 + 20 分钟打印/扫描/上传**；开卷，范围为截至递归定理（L11）的内容，**不含**数理逻辑那一节
Final Exam	25%	finals week 进行；开卷，只允许带教材与本课笔记

两点值得注意：

1. **quiz 占 25% 而不是 0**。Syllabus 明确写了这是学生反馈后的调整——"de-emphasize (but not eliminate) the midterm and final exams by adding graded live mini-quizzes for credit"。所以本课的通过策略不是"押期中期末"，而是"每讲都跟"。
2. **开卷的边界很窄**。Exams 页原文：只可用本课教材与你在讲座、复习课上记的笔记，"no other books or print-outs of other courses' problems"。这意味着一份自己整理的、把定理条件和归约方向写清楚的速查表，是合法且高回报的备考资产——这正是配套文件《18404_计算理论_项目上手速查.md》的用途。

0.5 26 讲地图

下表把 Calendar 页的讲次主题、Readings 页的教材章节、以及实际下载到的讲义 PDF 对齐。这是全笔记的索引，后面每一章都会标出对应的讲次。

讲次	主题（Calendar 页原文）	教材	讲义
1	Introduction, Finite Automata, Regular Expressions	§1.1	lec1
2	Nondeterminism, Closure Properties, Regular Expressions → Finite Automata	§1.2–1.3	lec2
3	The Regular Pumping Lemma, Finite Automata → Regular Expressions, CFGs	§1.4, §2.0–2.1	lec3
4	Pushdown Automata, CFG ↔ PDA	§2.2	lec4
5	The CF Pumping Lemma, Turing Machines	§2.3, §3.1	lec5
6	TM Variants, the Church–Turing Thesis	§3.2–3.3	lec6
7	Decision Problems for Automata and Grammars	§4.1	lec7
8	Undecidability	§4.2	lec8
9	Reducibility	§5.1	lec9
10	The Computation History Method	§5.2	lec10
11	The Recursion Theorem and Logic	§6.1–6.2	lec11
12	Time Complexity	§7.1	lec12
13	Midterm Exam	—	无
14	P and NP, SAT, Poly-time Reducibility	§7.2–7.3	lec14
15	NP-Completeness	§7.5	lec15
16	Cook–Levin Theorem	§7.4	lec16
17	Space Complexity, PSPACE, Savitch's Theorem	§8.1–8.2	lec17
18	PSPACE-Completeness	§8.3	lec18
19	Games, Generalized Geography	§8.3	lec19
20	L and NL, NL = coNL	§8.4	lec20
21	Hierarchy Theorems	§9.1	lec21
22	Provably Intractable Problems, Oracles	§9.2	lec22
23	Probabilistic Computation, BPP	§10.2	lec23
24	Probabilistic Computation (cont.)	§10.2	lec24
25	Interactive Proof Systems, IP	§10.4	lec25
26	coNP ⊆ IP	§10.4	lec26

几点结构性观察：

- **讲次顺序与教材顺序不一致**。L15（NP 完全性，§7.5）排在 L16（Cook–Levin，§7.4）之前，因为 Sipser 的讲法是先"假设 SAT 是 NP 完全"做归约练习，再回来补 Cook–Levin 的证明。做题时要注意这

个"先欠后还"的结构。

- **L13 没有讲义**，它就是期中考试。所以 26 讲只有 25 份 PDF。
- **教材覆盖范围**（Readings 页原文）：第 1、2 章（除 2.4）、3、4、5 章、6.1、7、8、9.1、9.2、10.2（除 primality 部分）、10.4 到 Theorem 10.33。**没讲**的内容包括：确定性上下文无关语言（2.4）、6.2 之后的可归约性进阶、9.3 电路复杂性、10.3 密码学。这决定了本笔记不把电路复杂性当正文，只在专题里做概览并标注「课外」。
- **三大板块的周数**（Syllabus 的 Course Outline）：自动机与语言理论 2 周、可计算性 3 周、复杂性 7 周。复杂性占了一半以上，这与很多学校"计算理论 = 自动机 + 停机问题"的印象不同。

0.6 材料形态与抓取口径

为了避免"二手转述"，这份笔记的所有技术内容都来自原始 PDF 的 `pdftotext -layout` 抽取。材料清单与实际核对结果：

材料	数量	说明
讲义 PDF	25	文件名形如 `MIT18_404f20_lecN.pdf`；每份 10–14 页幻灯片
视频逐字稿 PDF	25	文件名是 YouTube ID，首页标题形如 `MITOCW | 5. CF Pumping Lemma, Turing Machines`
Problem Set	6	hw1–hw6，题目全部指向教材第 2 版的习题号（Assignments 页原文说明）
Sample Exam	3	Fall 2001 期中、Fall 2006 期中、Fall 2006 期末
期末解答	1	Fall 2006 final solutions

一个必须说清楚的坑：**Assignments 页明确写着"The problem sets below were created for the Fall 2006 instance of this course"，并且所有题目都引自教材第 2 版**。所以 2020 年秋季实际布置的作业内容与页面上挂的这 6 份不完全相同（L1 讲义里提到的 Pset 1 是 "due in one week"，而页面上 hw1 的第 0.1 题是"读但不交"）。引用题号时我以 PDF 为准，并标注这是 2006 版题集。

0.7 与 CS170 的分工：为什么两门课都要学

同目录已有 `UCB_CS170_算法设计_学习笔记.md`，其中 NP 完全性占了 5–6 次课。两门课在这一块**表面重叠、实则互补**，分工可以这样切：

维度	CS170	18.404J
归约的目的	判定"这题没有多项式算法"，然后转去近似/随机/搜索	判定"这题根本不可判定"，或"这题是某复杂度类的完全问题"

维度	CS170	18.404J
归约的靶子	3-SAT、Independent Set、Vertex Cover、Hamiltonian Cycle	A_TM、HALT_TM、E_TM、PCP、TQBF、EQ_REX↑
归约的合法性标准	多项式时间	可计算（一般归约）或多项式时间（复杂性部分）
证明的重心	gadget 构造、正确性双向、复杂度核算	模型的形式定义、图灵机编码、对角线法、计算历史
典型作业	"证明 Max-Cut 是 NP 完全的"	"证明 E_TM 不可判定"、"证明 EQ_REX↑ 是 EXPSPACE 完全的"
复杂度类的处理	只用 P/NP，coNP 一笔带过	P、NP、coNP、PSPACE、L、NL、BPP、IP、EXPTIME/EXPSPACE 全讲，且证层次定理

具体地说，CS170 教会你"怎么打 gadget"，本课教会你"为什么 gadget 这套方法在可计算性层面会失效、以及复杂度类的层级本身可以被证明是严格的"。**本笔记刻意不重复 CS170 的算法设计细节**（近似比、随机化、线性规划对偶），只在必要处用一句话指回 CS170 的章节。

一个具体的衔接点：CS170 里 2-SAT 用 SCC 在线性时间解决；本课 L20 把 2SAT 证成 **NL 完全**——同一个问题，一个给出算法，一个给出它在空间层级里的精确位置。这种"同一个问题两种视角"的对照，是两门课一起学才有收益的地方。

0.8 建议的读法

按本课的结构，我建议三遍读：

1. **第一遍按讲次顺序读第 1–13 章**，把"形式定义 → 定理 → 证明模板"这条链走通。这一段的门槛在泵引理和归约方向，不在概念。
2. **第二遍读第 14–21 章的复杂性部分**，重点是记住"每个类用什么资源定义、完全性用哪种归约"这两张表，而不是记结论。
3. **第三遍用第 22–25 章的四个专题做横向压缩**，把散在各讲的闭包性质、泵引理、归约、类关系各自排成一张图。考前只看专题 + 第 27 章的自测题。

如果时间只够读一章，读第 10 章（对角线法）。它是整门课的方法论枢纽：A_TM 不可判定、Gödel 不完备、层次定理，三个看起来无关的结论用的是同一个动作——**把机器列成表，然后构造一个和每一行都不同的对象**。

第 1 章 有限自动机：把"记忆"压缩到有限个状态

> **对应材料**：Lec 1（Sipser §1.1） | 关键词：deterministic finite automaton, regular language

1.1 问题：一个只会"记状态"的机器能认多少语言

L1 用一个具体例子开场：设计一台机器，判断输入串里是否含有子串 `11`。给你的资源只有"当前处于三个状态中的哪一个"和"读下一个符号"这两个动作，没有计数器、没有栈、没有临时变量。于是问题变成：**有限的状态足够表达什么样的"历史"？**

答案不是"越多状态越强"，而是"状态数固定时，能区分的等价类数目固定"。这句直觉会在第 3 章的泵引理里变成严格的定理。

1.2 形式定义

定义（DFA，L1 原文）：一台确定性有限自动机（deterministic finite automaton, DFA）是一个五元组 $M = (Q, \Sigma, \delta, q_0, F)$：

成分	含义	约束
Q	状态集	有限
Σ	字母表	有限
δ	转移函数	$\delta : Q \times \Sigma \to Q$（**必须全定义且单值**）
q_0	起始状态	$q_0 \in Q$
F	接受状态集	$F \subseteq Q$

讲义里 L1 的例子上 $Q = \{q_1, q_2, q_3\}$、$\Sigma = \{0, 1\}$、$F = \{q_3\}$，转移表读作 $\delta(q_1, 0) = q_1$，$\delta(q_1, 1) = q_2$，$\delta(q_2, 0) = q_1$，$\delta(q_2, 1) = q_3$，$\delta(q_3, 0) = \delta(q_3, 1) = q_3$。

定义的边界：δ 是函数而不是关系，这是"确定性"的全部含义；δ 必须对每个 (q, a) 有定义，所以 DFA 读完输入一定停在某个状态。把这两条放松，就得到第 2 章的 NFA。

定义（接受的计算）：M 接受串 $w = w_1 \cdots w_n$，若存在状态序列 $r_0, r_1, \ldots, r_n$ 满足 $r_0 = q_0$、$r_i = \delta(r_{i-1}, w_i)$（$1 \le i \le n$）、$r_n \in F$。记 $L(M) = \{w \mid M \text{ 接受 } w\}$，称 M 识别（recognize）$L(M)$。

定义（正则语言）：语言 L 是正则的（regular），若某个有限自动机识别它。

1.3 定理与证明思路

定理（正则语言对并封闭）：若 L_1, L_2 正则，则 $L_1 \cup L_2$ 正则。

证明思路（乘积构造）：设 $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$、$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$。构造 $M = (Q, \Sigma, \delta, q_0, F)$，其中 $Q = Q_1 \times Q_2$、$q_0 = (q_1, q_2)$、$\delta\big((r_1, r_2), a\big) = \big(\delta_1(r_1, a), \delta_2(r_2, a)\big)$。关键是接受状态集：

$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

讲义在这里做了一个非常有教学价值的动作：**故意先写错**。L1 幻灯片原文标着 "$F = F_1 \times F_2$ NO! [gives intersection]"。写成笛卡尔积得到的是交集而不是并集，因为 $(r_1, r_2) \in F_1 \times F_2$ 要求两台机器同时接受。这个"先错后改"的设计说明一件事：乘积构造的难点不在乘积，而在**接受条件用哪个布尔运算**。

推论（对交、对补封闭）：改 $F = F_1 \times F_2$ 得交；把 F 换成 $Q \setminus F$ 得补。补运算要求 DFA 而不是 NFA——这是后面第 2 章"补运算是 NFA 的痛处"的伏笔。

1.4 例子

例 1（含 `11`）：三个状态足够：q_1 表示"还没见到 1"，q_2 表示"上一个符号是 1 但还没凑成 11"，q_3 表示"已经见过 11"。q_3 是吸收态（一旦进入就不再离开），这是"存在性条件"的通用设计模式。

例 2（偶数个 1）：两个状态，按 1 的个数奇偶分。讲义在 L1 留了一句 "make automaton for practice"。

1.5 常见误区

误区	为什么错	正确做法
"我画得出状态图，所以语言正则"	图只是构造，不是论证"存在一台机器"；漏画一个转移就不合法	给出五元组或完整转移表，确认 δ 全定义
把 ε 当成 Σ 里的符号	ε 是空串，长度 0，不是符号；DFA 不读它	Σ 里只放真实符号，ε 只在语言描述里出现
认为并集与交集的状态数不同	乘积构造对两者用的是同一套状态，只有 F 不同	记住"状态用乘积，接受条件用布尔运算"

第 2 章 非确定性、NFA 与三个等价性定理

> **对应材料**：Lec 2（Sipser §1.2–1.3）｜关键词：nondeterminism, subset construction

2.1 问题：并集已经闭包了，串接卡在哪

L1 结尾留下一个失败：证明 $L_1 L_2$ 正规时，我们想让机器"在 M_1 接受的位置切换到 M_2"，但**不知道在哪里切分输入**。M_1 接受的串长度不固定，切换点也不固定。

L2 的解法不是"更聪明的切分算法"，而是**改模型**：允许机器在切换点同时保留"继续留在 M_1"和"跳到 M_2"两种可能。这就是非确定性（nondeterminism）。

讲义把这一点讲得很清楚："Hold off. Need new concept."

2.2 形式定义

定义（NFA，L2 原文）：非确定性有限自动机（nondeterministic finite automaton, NFA）是五元组 $N=(Q,\Sigma,\delta,q_0,F)$，与 DFA 唯一的区别是

$$\delta: Q\times\Sigma_\varepsilon\to\mathcal{P}(Q),\qquad \Sigma_\varepsilon=\Sigma\cup\{\varepsilon\}$$

即：转移函数返回**状态集合**（$\mathcal{P}(Q)$ 是幂集），且允许读入 ε 而不消耗输入（"free move"）。

定义（NFA 的接受）：N 接受 w，若存在一条从 q_0 到某个 $f\in F$ 的路径，其非 ε 标签依次拼成 w。注意"存在"两个字——只要**某一条**路径走到接受态即算接受。

L2 给了三种理解非确定性的方式，这三种说法在考试里都要能用：

视角	说法	适合用来做什么
计算（computational）	每一步分叉出新线程，任一分支接受则接受	解释为什么"猜"不花钱
数学（mathematical）	计算过程是一棵树，只要有一个叶子在接受态即接受	写形式证明、做归纳
魔法（magical）	每一步都**猜对**该往哪走	设计算法（第 16 章 SAT、第 20 章 NL 全靠这个）

2.3 定理与证明思路

定理（NFA 不比 DFA 强）：若 NFA N 识别 L，则 L 正则。

证明思路（子集构造, subset construction）：让 DFA M 的状态是 NFA 状态的**集合**，即"读完当前前缀后 NFA 所有可能停在的状态"。

成分	构造
Q'	$\mathcal{P}(Q)$
q_0'	$\{q_0\}$
$\delta'(R,a)$	$\bigcup_{r\in R}\delta(r,a)$
F'	$\{R\in Q' \mid R\cap F\neq\varnothing\}$

注意 F' 用的是**交非空**，不是 $R\subseteq F$——这是全章最容易写错的一行，它的含义是"集合里有一个是接受态就接受"，正对应 NFA 的"存在一条路径"。

规模：N 有 k 个状态时 M 至多 2^k 个状态。L2 的 check-in 就是在问这个（答案 2^k，选项里 k^2、$2k$ 是陷阱）。

定理（正则语言对串接、星号封闭）：由 NFA 直接给出构造，不需要"猜切分点"：

- **串接 L_1L_2**：把 M_1 的每个接受态加一条 ε 边指向 M_2 的起始态；M_1 的接受态不再接受。

- **星号 L^***：加一个新起始态（兼接受态），从它 ε 到原起始态；从每个原接受态 ε 回原起始态，并 ε 到新接受态。**必须单独处理 $\varepsilon \in L^*$**——L2 幻灯片专门写了 "Make sure M' accepts ε"。

定理（正则表达式 → NFA）：对正则表达式的结构做归纳。原子情形 $a \in \Sigma$、ε、$\varnothing$ 各给一台小机器；复合情形 $R_1 \cup R_2$、$R_1 \circ R_2$、R_1^* 直接调用上面三个闭包构造。因为正则表达式是有限语法树，归纳一定终止。

2.4 例子

例（$a \cup ab^*$ 的 NFA）：L2 的板书例子。先给 `a` 一台机器，再给 `b` 一台，用 ε 分叉实现并集，用 ε 串接实现 ab^*。构造出来的机器状态数比"最小 DFA"多，这没关系——**构造只要求正确，不要求最优**。

例（aab 在 NFA 上的两条路径）：L2 的 check-in 2.1 用 `aab` 追问接受与否，答案是"接受"，因为存在一条走通的路径。这个小题的目的是打破"非确定性机器必须每条路都对"的误解。

2.5 常见误区

误区	为什么错	正确做法
"NFA 比 DFA 强"	子集构造给出 DFA，能力完全相同	非确定性带来的是**描述简洁**，不是**计算能力**
子集构造里漏掉 ε-闭包	$\delta'(R, a)$ 应先把 R 沿 ε 边膨胀再走 a	写代码/写证明时先算 $E(R)$ 再算 $\bigcup \delta(r, a)$
把 F' 写成 $R \subseteq F$ 或 $R = F$	语义是"存在接受态"，不是"全是接受态"	$F' = \{R \mid R \cap F \neq \varnothing\}$

第 3 章 泵引理：如何证明"不存在这样的自动机"

> **对应材料**：Lec 3（Sipser §1.4, §2.0–2.1） | 关键词：pumping lemma, GNFA, pigeonhole principle

3.1 问题：否证比构造难一个量级

证明一个语言正则，只需给出一台机器。证明它不正则，需要排除**无穷多台**机器。L3 把难点说得很直白：**"It is not enough to say that you couldn't find a DFA for it."**

L3 还给了一个专门打脸的例子，值得记住：

$$J = \{w \mid w \text{ 中 } 01 \text{ 与 } 10 \text{ 作为子串出现次数相等}\}$$

直觉说"DFA 不能无界计数，所以 J 不正则"。**但 J 是正则的**。原因：01 与 10 的出现次数之差只取决于首尾两个符号，是一个有界量——$\#01 - \#10 \in \{-1, 0, 1\}$，且只在串首尾符号不同的时候为 ± 1。所以三四个状态就够。L3 的原话是 "Sometimes intuition works, but it can also be wrong. Moral: You need to give a proof."

这一页的教学价值高于任何定理：**直觉负责猜，证明负责定**。

3.2 形式定义

定理（泵引理, pumping lemma for regular languages）：对每个正则语言 L，存在常数 p（泵长度, pumping length），使得若 $s \in L$ 且 $|s| \geq p$，则 s 可写成 $s = xyz$ 且满足

条件	内容
1	对**所有** $i \geq 0$，$xy^iz \in L$
2	$
3	$

三个条件的角色完全不同，考试里最常被混掉的就是这条分工：

- 条件 1 是**结论**（用来导出矛盾）；
- 条件 2 保证"泵"这个动作真的改变串；
- 条件 3 是**限制**（用来把 y 锁在前 p 个符号里，从而控制 x, y 的形状）。

3.3 定理与证明思路

证明（鸽笼原理）：设 DFA M 识别 L，M 有 p 个状态。取 $s \in L$、$|s| \geq p$，令 $s = s_1 \cdots s_n$（$n \geq p$）。考虑 M 读 s 时经过的状态序列 $r_0, r_1, \ldots, r_n$，共 $n+1 \geq p+1$ 个状态，而状态集只有 p 个元素，所以必有 $0 \leq i < j \leq p$ 使 $r_i = r_j$。取 $x = s_1 \cdots s_i$、$y = s_{i+1} \cdots s_j$、$z = s_{j+1} \cdots s_n$。

- $|y| = j - i > 0$，得条件 2；
- $j \leq p$ 得 $|xy| = j \leq p$，得条件 3；
- $r_i = r_j$ 意味着从 r_i 读 y 会回到 r_i，所以可以把这个环走任意次，得条件 1。

L3 的 check-in 3.2 问"这个证明用了什么"，答案是**鸽笼原理**（pigeonhole principle）——幻灯片把它和 Burnside 计数、Coronavirus 计算并列当干扰项。这个 check-in 问的其实是"你是否知道证明的支点在哪"。

定理（DFA → 正则表达式，GNFA 方法）：L3 的另一半。引入广义非确定性有限自动机（generalized NFA, GNFA）：边上可以贴**正则表达式**而不只是符号。规范化形式要求：唯一的接受态、唯一的起始态、除"只出起始态"和"只进接受态"之外每对状态之间都有一条边（可以贴 $\varnothing$）。

引理：每个 GNFA 都有等价的正则表达式。对状态数 k 归纳：

- 基础 $k = 2$：只有一个转移标签，直接就是 R；
- 归纳步：任选一个既非起始也非接受的 q_{rip}，删掉它，并对每一对 (q_i, q_j) 把标签改成

$$R_{ij} \cup R_{i,\text{rip}} \left(R_{\text{rip},\text{rip}}\right)^* R_{\text{rip},j}$$

即"原来直达的路径"并上"绕经 rip 的路径"。状态数从 k 降到 $k-1$，归纳成立。

这一步的 check-in 3.1 很有意思：幻灯片问"我们刚证了 GNFA→RE，怎么完成 DFA→RE 的目标？"答案是 **"(c) We are already done. DFAs are a special type of GNFAs."** 也就是说 DFA→RE 不需要新构造，只需要

观察"DFA 是 GNFA 的特例"。这类"用特例关系省掉一个构造"的技巧在理论课里反复出现。

3.4 例子

例 1（$\{0^n1^n \mid n \geq 0\}$ 不正则）：假设正则，取泵长度 p，选 $s = 0^p1^p$。由条件 3，xy 全在 0 段内，故 $y = 0^k$（$k \geq 1$）。于是 $xy^2z = 0^{p+k}1^p \notin L$，与条件 1 矛盾。

例 2（$\{ww \mid w \in \Sigma^*\}$ 不正则）：L3 的关键教学点在这里——**不能选 $s = 0^p0^p$**，因为那个串"能被泵且仍留在语言里"（$x = \varepsilon, y = 00, z = 0^{p-2}0^p$ 之类的切分可以成立）。正确选法是 $s = 0^p10^p1$，此时 y 落在第一段 0 里，泵一次后两半就不再相等。幻灯片原文："Try $s = 0^p0^p \in L$. But that s can be pumped and stay inside L. **Bad choice.**"

3.5 常见误区

误区	为什么错	正确做法
用泵引理证明某语言**正则**	泵引理是必要条件，不是充分条件；满足它的语言可以不正则	证正则只给 DFA/NFA/RE；泵引理只用来否证
自己"挑"切分 xyz	条件是对**存在**的 p、**任意的**合法切分都要成立	反证时你只能控制 s，切分由对手给，你只需证明每种切分都矛盾
忽略条件 3 导致 y 可以跨段	不锁 $	xy

第 4 章 上下文无关文法：从"状态"升级到"栈"

> **对应材料**：Lec 3 末–Lec 4（Sipser §2.0–2.1） | 关键词：context-free grammar, derivation, ambiguity

4.1 问题：正则表达式写不出嵌套

正则语言有一个本质限制：它能表达的模式是"局部"的，不能表达"配对的括号"这种需要**无界嵌套**的结构。$\{0^n1^n\}$ 不正则（第 3 章），但它的描述只需要一句话："先是 n 个 0，再是 n 个 1"。

n 这个"同一个数出现两次"的约束，等价于要求"记住已经读了多少"，而这是有限状态做不到的。文法的做法不是加内存，而是**允许定义递归地引用自己**：

$$S \to 0S1 \mid \varepsilon$$

这就是上下文无关文法（context-free grammar, CFG）的全部魔法：`S` 出现在自己的右部，于是"0 和 1 必须配对出现"被编码进了产生式的形状里。

4.2 形式定义

定义（CFG，L4 原文）：上下文无关文法是一个四元组 $G = (V, \Sigma, R, S)$：

成分	含义	约束
V	变量（variables，也叫非终结符）	有限
Σ	终结符（terminals）	有限，且 $V \cap \Sigma = \varnothing$
R	规则集	每条规则形如 $A \to w$，其中 $A \in V$，$w \in (V \cup \Sigma)^*$
S	起始变量	$S \in V$

"上下文无关"这个词的含义：规则左部**只能是一个变量**。对比上下文有关文法（context-sensitive grammar）允许 $uAv \to uwv$。课程不覆盖后者，但知道这个对比能记住定义的名字来源。

定义（推导与生成）：

- $uAv \Rightarrow uwv$ 若 $A \to w$ 是规则（一步推导）；
- $\Rightarrow^*$ 是 $\Rightarrow$ 的自反传递闭包（多步推导）；
- $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$，称为上下文无关语言（context-free language, CFL）。

定义（歧义, ambiguity）：若存在串 $w \in L(G)$ 有**两棵不同的分析树**（parse tree），则称 G 是歧义的。L4 特别强调"不同分析树"，不是"不同推导顺序"——同一个分析树可以对应多种替换顺序，那不算歧义。

4.3 定理与证明思路

定理（每个正则语言都是 CFL）：两条路都通：（a）DFA → CFG 的直接构造（把状态变成变量，$A_i \to aA_j$）；（b）DFA → PDA → CFG。L4 的 check-in 4.3 就在问这个（"Is every Regular Language also a Context Free Language?" 答 Yes），而 L4 结尾的回忆图把包含关系画成了**嵌套矩形**：regular ⊂ context-free。

定理（CFG → PDA）：见第 5 章。

观察（歧义是文法属性，不是语言属性）：L4 的对照实验是全课最漂亮的教学设计之一：

文法	规则	是否歧义
G_2	$E \to E+T \mid T$，$T \to T \times F \mid F$，$F \to (E) \mid a$	**无歧义**
G_3	$E \to E+E \mid E \times E \mid (E) \mid a$	**有歧义**

两者生成**同一个语言**（$L(G_2) = L(G_3)$），但 G_3 里 $a + a \times a$ 有两棵分析树。G_2 之所以无歧义，是因为它把"优先级"编码进了**变量分层**：E 管加法、T 管乘法、F 管原子。这说明**歧义可以被文法设计消除**，而"语言本身是否固有歧义（inherently ambiguous）"是另一个问题（存在固有歧义的 CFL，课程不证）。

4.4 例子

例 1（算术表达式）：L4 的核心例子。$E \to E+T \mid T$、$T \to T \times F \mid F$、$F \to (E) \mid a$，生成 `a+a×a`、`(a+a)×a` 等。幻灯片指出：**分析树额外携带了"× 优先于 +"的信息**。这句话解释了为什么编译器前端要用文法而不是正则表达式。

例 2（$\{0^n 1^n \mid n \geq 0\}$）：$S \to 0S1 \mid \varepsilon$。推导 `S ⇒ 0S1 ⇒ 00S11 ⇒ 0011`。

例 3（回文）：$S \to 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$。这是"中心对称"结构的标准写法。

4.5 常见误区

误区	为什么错	正确做法
规则左部可以写成串（如 $aB \to ab$）	那就成了上下文有关文法	CFG 左部必须是**单个变量**
把"两种推导顺序"当歧义	同一棵树的不同展开顺序不算	歧义判据是**不同的分析树**，等价于存在串有两个不同的最左推导
认为无歧义文法唯一	同一语言可以有很多无歧义文法	只问"是否存在"，不问"是否唯一"

第 5 章 下推自动机与 CFG↔PDA 的等价

> **对应材料**：Lec 4（Sipser §2.2）｜关键词：pushdown automaton, stack, nondeterminism

5.1 问题：栈到底买到了什么

L4 的对照图把四种对象排成两列：**识别器**（recognizer）与**生成器**（generator）。正则语言这一行是 DFA/NFA 对正则表达式；上下文无关语言这一行是 PDA 对 CFG。这张表的含义是：**每一类语言都有"认"和"生"两种等价的描述方式**，而这一章要补的是"PDA 认 = CFG 生"。

栈带来的能力可以一句话说清：它允许**后进先出**的无界存储。后进先出恰好匹配"嵌套"——最后打开的括号必须最先关闭。这也解释了为什么栈不够用来识别 $\{ww\}$：那需要随机访问前半段，而栈只能倒着读。

5.2 形式定义

定义（PDA，L4 原文）：下推自动机（pushdown automaton, PDA）是六元组 $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$：

成分	含义
Q	状态集（有限）
Σ	输入字母表
Γ	栈字母表（stack alphabet）
δ	$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$
q_0	起始状态
F	接受状态集

转移 $\delta(q, a, c) \ni (q', d)$ 读作："在状态 q、输入下一个符号是 a、栈顶是 c 时，可以转到 q' 并把栈顶 c 换成 d。"当 $d = \varepsilon$ 时是弹栈（pop），$c = \varepsilon$ 时是压栈（push），$a = \varepsilon$ 时是不消耗输入。

三个"ε"必须分清，这是本章唯一的记忆负担：

- Σ_ε 里的 ε：**不读输入**；
- Γ_ε 里的第一个 ε（输入的 c）：**不看栈顶**；
- Γ_ε 里的第二个 ε（输出的 d）：**弹栈**。

定义（接受）：PDA 接受 w，若存在一条从 $(q_0, w, \text{空栈})$ 出发的计算路径，在**读完整个输入后**进入接受态（或按"栈空即接受"的约定栈空）。L4 的两个例子都注明 "acceptance only at end of input"——这个约定必须在构造时守住，否则会接受不该接受的串。

5.3 定理与证明思路

定理（CFL → PDA）：若 L 是 CFL，则某台 PDA 识别 L。

证明思路（"猜替换 + 栈上模拟最左推导"）：给定 CFG G，构造 PDA 如下：

1. 把起始变量压栈；
2. 循环：若栈顶是**变量**，非确定性地选一条该变量的规则，用右部替换栈顶；若栈顶是**终结符**，弹出它并与下一个输入符号比较，不等则拒绝；
3. 栈空则接受。

L4 在这里插了一个非常关键的"防作弊"说明：**"Problem! Access below the top of stack is cheating!"** 最初的直觉是"把中间串存在栈上，然后逐个比对"，但那要求读栈的第二个元素——栈不支持。修正办法是**只在栈顶是变量时才替换**，于是栈永远只在顶端增长和收缩，模拟的是**最左推导**（leftmost derivation）。

定理（PDA → CFG）：L4 明确说 "In book. You are responsible for knowing it is true, but not for knowing the proof." 也就是说：结论必须记住（CFL = PDA 可识别的语言），证明不考。这是一个诚实的课程取舍，也提醒我们**不要在这上面花时间**。

推论（两个高价值推论）：

1. 每个正则语言都是 CFL（DFA 是 PDA 的特例）；

2. 若 L 是 CFL、R 是正则，则 $L \cap R$ 是 CFL。**证明思路（L5 原文）**：让 L 的 PDA 在有限控制里同时模拟 R 的 DFA，两个模型"并联"运行。这条推论是 hw1 第 0.1 题（教材 2.18）的内容：$\mathbf{CFL} \cap \mathbf{regular} = \mathbf{CFL}$。

注意推论 2 不能被推广：两个 CFL 的交可以不是 CFL（第 6 章），所以 CFL 类对 ∩ 不封闭。

5.4 例子

例 1（$\{0^n 1^n \mid n \geq 0\}$）：读 0 就压栈，读到 1 就开始弹，读完时栈空则接受。三步：压、弹、判空。

例 2（$\{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$，偶数长度回文）：读入并压栈，**非确定性地**在某个位置切换到"弹栈比对"模式。L4 指出 "This language requires nondeterminism"——确定性的 PDA 识别不了它（这是课外事实，课程只做提示）。

5.5 常见误区

误区	为什么错	正确做法
让 PDA 访问栈的第二个元素	模型不允许，栈只有栈顶可读写	改写算法，只在栈顶操作（最左推导）
用 PDA 识别 $\{ww\}$	需要随机访问前半段	只有回文 $ww^{\mathcal{R}}$ 能用栈；$\{ww\}$ 不是 CFL（第 6 章证）
混淆"栈空接受"与"接受态接受"	两种约定等价但构造时不能混用	选定一种，全文统一；课程例子用"接受态 + 读完输入"

第 6 章 CFL 泵引理：五段分解与"剪贴论证"

> **对应材料**：Lec 5（Sipser §2.3）｜关键词：pumping lemma for CFLs, parse tree, cutting and pasting

6.1 问题：为什么正则那套泵法不够用

正则泵引理把串切成三段 xyz，因为 DFA 的计算是一条**路径**，重复一段路径就得到环。CFL 的计算是一棵**树**：串是树叶从左到右读出来的，重复"一段路径"对应的是**重复两段子树**，所以分解要切成五段。

这就是为什么 CFL 泵引理的结论长成 $s = uvxyz$ 而不是 xyz：v 与 y 是两棵可以同时被"复制"的子树。

6.2 形式定义

定理（CFL 泵引理，L5 原文）：对每个上下文无关语言 L，存在常数 p，使得若 $s \in L$ 且 $|s| \geq p$，则 s 可写成 $s = uvxyz$ 且满足

条件	内容	与正则版的对应
1	对所有 $i \geq 0$，$uv^i xy^i z \in L$	对应 $xy^i z \in L$，但**同时**泵 v 与 y
2	$\|vy\| > 0$	对应 $\|y\| > 0$，两段至少一段非空
3	$\|vxy\| \leq p$	对应 $\|xy\| \leq p$，中间三段被限长

为什么是 $|vxy| \leq p$：vxy 对应"最低的那个重复变量所覆盖的叶子"，它的长度受"最大分支度 l 与变量数 b"控制，与 p 同阶。而 x 在中间，是两次重复之间的"间隔"。

6.3 定理与证明思路

证明（剪贴论证, cutting and pasting）：设 G 是 L 的 CFG，b 为变量个数，l 为最长规则右部的长度（即分析树的最大分支度）。

1. 取 $s \in L$、$|s| \geq p$，取 s 的**最小**分析树。
2. 若树高 h，则叶子数至多 l^h，故 $|s| \leq l^h$。
3. 取 $p = l^{b+1}$。若 $|s| \geq p > l^b$，则 $h > b$，即最长路径上有 $b+1$ 个变量。
4. 由鸽笼原理，某个变量 R 在同一条路径上**重复出现**两次。
5. 取**最低**的那一对重复 R：上层的 R 生成 vxy，下层的 R 生成 x。于是把下层子树"剪掉"得到 uxz，把上层子树"贴进"下层位置得到 uv^2xy^2z，一般地得到 uv^ixy^iz。
6. 因为取的是最低重复，vxy 的生成树高度受控，得 $|vxy| \leq p$；又 R 的两次出现不同（否则不是最小树），故 $|vy| > 0$。

L5 的幻灯片把这一段画成"长串 → 高树 → 重复变量 → 剪贴"，并注明"start with the smallest parse tree for s"和"pick the lowest repetition of a variable"。这两句话是证明能成立的技术核心，缺一个都证不下去。关于 p 的具体形式：L5 取 $p = l^{b+1}$；Sipser 书里取的是 $b^{|V|+1}$ 一类的常数（不同版本略有差异）。**两者只是常数不同，不影响任何应用**——泵引理只要求"存在 p"，具体多大无关。这一点在考试里不必纠结。

6.4 例子

例 1（$\{0^n1^n2^n \mid n \geq 0\}$ 不是 CFL）：这是 L5 的主例。假设 CFL，取 $s = 0^p1^p2^p$。由条件 3，vxy 长度不超过 p，所以它**不可能同时包含 0、1、2 三种符号**（跨度至少 $p+2$）。于是 vxy 至多覆盖两种符号，泵一次后三种符号的个数不再相等，$uv^2xy^2z \notin L$，矛盾。

这一步的技术要点：**条件 3 的作用是限制 vxy 的跨度**，从而保证"泵"会破坏某种计数平衡。如果不写条件 3，这个论证无法进行——这是学生最常漏的一步。

例 2（$\{ww \mid w \in \{0,1\}^*\}$ 不是 CFL）：L5 强调**串的选择**。先试 $s = 0^p10^p1$（"容易被泵"），失败；改用 $s = 0^p1^p0^p1^p$。论证：条件 3 保证 vxy 不会同时跨过两段 0 或两段 1，故泵之后某两段 0（或 1）长度不等，串不再是 ww 形式。

幻灯片上的原话是"Condition 3 implies that vxy does not overlap two runs of 0s or two runs of 1s"，这一步需要**数一数 s 的分段结构**才能确认，属于"选对串之后还要说清楚为什么"的典型。

6.5 常见误区

误区	为什么错	正确做法
只泵 v 或只泵 y	条件是 uv^ixy^iz，两段必须**同时**泵	写作时统一写 i，不要给两段不同的指数
忘记 $\|vxy\| \le p$	少了它就无法限制 vxy 的跨度，论证崩掉	论证第一句就写"由条件 3，vxy 至多覆盖…"
认为 x 必须非空	$\|vy\| > 0$ 只约束 v, y；x 可以为空	x 是"间隔"，可空

第 7 章 图灵机：把"计算"变成一个数学对象

> **对应材料**：Lec 5（Sipser §3.1） | 关键词：Turing machine, configuration, decider

7.1 问题：自动机加了什么才够"通用"

L5 在讲完 CFL 泵引理后直接跳到图灵机（Turing machine, TM），中间没有过渡。这个跳跃的合理性在于：前四章的所有模型都只有**只读输入 + 有限控制 + 一个有界/后进先出的存储**。要让模型"通用"，需要三件事：

1. 能**写**（不只是读）；
2. 头能**双向**移动（不只是从左到右）；
3. 存储**无界**（不是栈那样的受限访问）。

L5 把这三条连同"无穷多空白符"和"可以在任意时刻接受/拒绝"一起列为 TM 的五个特征。最后一条容易被忽略但很关键：DFA 只能在读完输入时给答案，TM 可以中途停机。

7.2 形式定义

定义（TM，L5 原文）：图灵机是七元组 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$：

成分	含义	约束
Q	状态集	有限
Σ	输入字母表	不含空白符 ⊔
Γ	带字母表（tape alphabet）	$\Sigma \subseteq \Gamma$，$\sqcup \in \Gamma$
δ	$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$	**函数**（本模型是确定性的）
q_0	起始状态	
q_{acc}	接受状态	
q_{rej}	拒绝状态	$q_{acc} \neq q_{rej}$

定义（格局, configuration）：三元组 $(\text{状态}, \text{头位置}, \text{带内容})$，即某时刻的完整快照。第 12 章证明计算历史时会把格局编码成字符串。

定义（TM 的三种结局）：对输入 w，M 可能

1. 接受（进入 q_{acc}）；
2. 拒绝并停机（进入 q_{rej}）；
3. **不停机（loop）**。

第三种结局是 TM 与 DFA/NFA 的本质差别，也是整个可计算性理论的源头：它使"识别"与"判定"分家。

定义（识别与判定）：

- $L(M) = \{w \mid M \text{ 接受 } w\}$；称 M **识别**（recognizes）$L(M)$；
- 若 M 在所有输入上都停机，称 M 是**判定器**（decider）；此时称 M **判定**（decides）$L(M)$；
- L 是**图灵可识别**的（Turing-recognizable），若某 TM 识别它；L 是**图灵可判定**的（Turing-decidable），若某判定器判定它。

L5 的包含图画成三层嵌套：regular ⊂ CFL ⊂ T-decidable ⊂ T-recognizable。

7.3 定理与证明思路

定理（存在识别 $\{a^n b^n c^n \mid n \geq 0\}$ 的 TM）：这是 L5 的主例，算法分六步：

1. 向右扫描到空白，途中检查输入是否属于 $a^* b^* c^*$，否则拒绝；
2. 头回到最左端；
3. 扫描并"划掉"一个 a、一个 b、一个 c；
4. 若三类符号各剩最后一个，接受；
5. 若某类已空而其他类未空，拒绝；
6. 若都还有剩余，回到步骤 3。

技术要点（"划掉"怎么实现）：L5 的 check-in 5.2 专门问这个。答案是 **(b) 用带字母表** $\Gamma = \{a, b, c, \dot{a}, \dot{b}, \dot{c}, \sqcup\}$——即给每个符号加一个"已划掉"的变体，而不是给模型加橡皮擦。这个细节很重要：

模型的能力不靠加特性获得，而靠字母表的编码获得。同一个思想在第 8 章（多带→单带用"点符号"标记头位置）和第 16 章（Cook–Levin 用带符号编码格局）反复出现。

证明思路的特征：这类证明不需要写出转移表。L6 的记法约定是

$$M = \text{“On input } w\text{: [English description of the algorithm]”}$$

即"高级描述"就够，因为"in principle"总能展开成状态与转移（L6 原话 "High-level description is ok"）。这条约定是全课证明可读性的基础，也是考试里唯一被接受的写法。

7.4 例子

例（$\{ww\}$ 可被 TM 识别）：DFA 不行、PDA 不行、TM 可以（来回扫描做标记、逐位比对两半）。这是"模型层级"的直观证据：TM 严格强于 PDA。

7.5 常见误区

误区	为什么错	正确做法
认为 TM 一定停机	定义允许 loop，这是可计算性理论的根基	区分"识别"（可 loop）与"判定"（必停机）
把"拒绝"等同于"停机拒绝"	拒绝有两种：停机拒绝与不停机	写证明时说明是哪种；讨论可判定性时只关心"是否停机"
用"加橡皮擦""加计数器"扩展模型	模型能力靠编码获得	用带字母表的变体符号实现"划掉"

第 8 章 TM 变体、丘奇–图灵论题与编码记号

> **对应材料**：Lec 6（Sipser §3.2–3.3）｜关键词：multi-tape TM, NTM, enumerator, Church–Turing thesis

8.1 问题：凭什么"这一个模型"代表所有计算

如果图灵机只是众多计算模型中的一个，那么关于它的定理凭什么说是"关于计算的"？L6 的策略是**证明所有合理模型互相等价**，然后把等价性提升为一条论题（thesis）。

这是全课唯一的"非定理"。理解它的地位，比背它更重要。

8.2 形式定义

定义（多带图灵机, multitape TM）：有 k 条带，每条带各有读写头，初始时第 1 条带放输入、其余空白；一步内可以读写每条带并各自左右移动。转移函数

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R\}^k$$

定义（非确定性图灵机, NTM）：

$$\delta : Q \times \Gamma \to \mathcal{P}\big(Q \times \Gamma \times \{L, R\}\big)$$

计算形成一棵树，只要**某个分支**接受则接受。

定义（图灵枚举器, Turing enumerator）：一台带打印机的确定性 TM，从空白带开始，可以打印出串 $w_1, w_2, \ldots$（可能无穷）。它**生成**语言而不识别语言：$L(E) = \{w \mid E \text{ 打印 } w\}$。

记号约定（L6 的核心贡献之一）：

记号	含义
$\langle O \rangle$	对象 O（多项式、自动机、图…）的字符串编码
$\langle O_1, O_2, \ldots, O_k \rangle$	多个对象一起编码成单个串
$M =$ “On input w: [English]”	用高级描述写 TM

L6 的 check-in 6.3 问"$\langle x, y \rangle$ 是不是一个好选择"，答案是 Yes——编码是**任意但固定**的单射即可，不同编码之间可以多项式时间互转。

8.3 定理与证明思路

三条等价性定理，证明结构完全同构：**"模拟"**。

定理 1（多带 → 单带）：把 k 条带的内容存在一条带上，用 `#` 分隔，用"点符号"（dotted symbol）标记各读写头位置；模拟一步需全带扫描找点符号 → 再扫描更新 → 必要时搬移。代价是单带模拟一步 $O(n)$ 步，第 12 章会算成 $O(t^2)$ 放大。

定理 2（NTM → 确定性 TM）：确定性 TM 用**广度优先**遍历计算树；每条分支的带存在自己的 block 里，分叉时复制块，任一分支接受则接受。**注意只保持了可识别性，没保持时间**——NTM 转 DTM 是指数级，这正是 P vs NP 的根源；L6 刻意只谈可识别性，NTIME 留到第 15 章。

定理 3（枚举器 ↔ 识别器）：L 图灵可识别 iff $L = L(E)$ 对某枚举器 E。

两个方向：

- (←) 给定 E，造 TM M：对输入 w 模拟 E，每当 E 打印 x 就检查 $x = w$，相等则接受。**这不需要 M 停机**，只要"打印出来就能认"。
- (→) 给定 M，造 E：**这里有个陷阱**。天真的做法是按字典序枚举 Σ^* 的每个串 s_i 并模拟 M，接受就打印。问题是：若 M 在某个 s_i 上 loop，E 就卡死，后面的串永远打印不出来。修正：**对 $i = 1, 2, \ldots$，让**

M 在 $s_1, \ldots, s_i$ 上各跑 i 步，接受的就打印。这就是"时间片轮转"（dovetailing）。

L6 的 check-in 6.1 问"枚举器是否总按字典序打印"，答案 **No**——因为 dovetailing 会打乱顺序。这个小问题问的其实是"你是否理解 dovetailing 的代价"。

丘奇-图灵论题（Church–Turing Thesis）：

$$\text{算法（直观）} \quad = \quad \text{图灵机（形式）}$$

L6 明确它的身份：这不是定理，是**定义性论题**（"Instead of Turing machines, can use any other 'reasonable' model"）。可替代的模型包括 λ 演算、RAM、你喜欢的编程语言。论题的力量在于：任何一次"我设计了一个算法"的断言，都可以换成"存在一台 TM"。

希尔伯特第十问题（Hilbert's 10th Problem）：1900 年希尔伯特提出 23 个问题，第 10 问是"给出判定丢番图方程是否有整数解的算法"。1970 年 Matiyasevich 证明**不可判定**（L6 幻灯片写 1970；L10 幻灯片在回顾时写 1971，属讲义口径差异，学界通常记 1970 年）。

$$L = \{\langle p \rangle \mid \text{多项式 } p(x_1, \ldots, x_k) = 0 \text{ 有整数解}\}$$

这个例子在课上的作用是：**它看起来完全是数论问题，与机器无关，却是不可判定的**——这是"不可判定性不是人造怪物"的最强证据。L6 还补了一句："L is T-recognizable"（因为可以枚举整数元组逐个试）。

8.4 例子

例（"合理模型"的边界）：并非所有模型都等价。带计数器的自动机等价于 PDA（弱于 TM）；而允许无限精度实数的模型则"免费"算出了不可计算函数（强于 TM）。课程不讨论这些，但知道边界的形状有助于理解论题的地位——它排除的是"不合常理"的模型，不是"所有能想象的模型"。

8.5 常见误区

误区	为什么错	正确做法
把丘奇-图灵论题当定理去证	它连接直觉概念与形式概念，不可证	引用它时说"由论题"而不是"由定理"
认为 NTM 转 DTM 只慢常数倍	是指数级（$2^{O(t)}$）	可识别性保持，时间不保持
枚举时按字典序直接模拟	遇到 loop 就卡死	用 dovetailing（时间片轮转）

第 9 章 自动机与文法的判定问题

> **对应材料**：Lec 7（Sipser §4.1）｜关键词：A_DFA, E_DFA, EQ_DFA, A_CFG, E_CFG

9.1 问题：把"问题"变成"语言"

第 7 章把计算变成了数学对象，第 9 章把**问题**变成了数学对象。做法是：把"关于对象 O 是否成立"编码成语言 $\{\langle O\rangle \mid \text{性质成立}\}$。这一步是全课程最重要的一次抽象转换——一旦"问题"成了语言，"可解"就成了"可判定"。

L7 列出的七个问题构成了一个"能力阶梯"：

语言	问题	结论
A_{DFA}	DFA 是否接受某串	可判定
A_{NFA}	NFA 是否接受某串	可判定
E_{DFA}	DFA 的语言是否为空	可判定
EQ_{DFA}	两个 DFA 是否等价	可判定
A_{CFG}	CFG 是否生成某串	可判定
E_{CFG}	CFG 的语言是否为空	可判定
EQ_{CFG}	两个 CFG 是否等价	**不可判定**
ALL_{CFG}	CFG 是否生成 Σ^*	**不可判定**
A_{TM}	TM 是否接受某串	**不可判定**（但可识别）

9.2 形式定义

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ 是 } \mathbf{DFA} \text{ 且 } B \text{ 接受 } w\}$$

其余依此类推。注意编码记号 $\langle\cdot\rangle$ 的使用：**问题实例永远是字符串**，不是"图"或"文法"本身。

9.3 定理与证明思路

定理（A_{DFA} 可判定）：$M_{A\text{-DFA}}$ = "On input $\langle B, w\rangle$：1. 检查格式合法，否则拒绝；2. 在 w 上模拟 B；3. 停在接受态则接受，否则拒绝。"正确性依赖：模拟只需记住"当前状态 + 头位置"（有限信息），步数恰为 $|w|$，故必停机。

定理（A_{NFA} 可判定）：先转等价 DFA N'，再调用 $M_{A\text{-DFA}}$。**这是全课第一个"子程序"模式**——L7 原话 "Use conversion construction and previously constructed TM as a subroutine"，后面所有可判定性证明都是它的变体。

定理（E_{DFA} 可判定）：$M_{E\text{-DFA}}$ = "On input $\langle A\rangle$：1. 标记起始态；2. 重复直到无新状态：标记所有从已标记状态有入边的状态；3. 若无接受态被标记则接受，否则拒绝。"**注意方向**：从起点**正向**可达；写反成"从接受态反向"也成立，但必须与"起点"或"接受态"之一对齐，不能混用。

定理（EQ_{DFA} 可判定）：构造对称差 DFA

$$L(B) = \big(L(A) \cap \overline{L(C)}\big) \cup \big(\overline{L(A)} \cap L(C)\big)$$

然后跑 E_{DFA}：$L(A) = L(C)$ 当且仅当 $L(B) = \varnothing$。

L7 的 check-in 7.3 问"为什么不能用同样的技巧证 EQ_{CFG} 可判定"，答案是 **(c) CFL 对补和对交不封闭**。这是本章最重要的一个"技术为什么失效"的问题：对称差构造要求闭包性质，而 CFL 没有。

定理（A_{CFG} 可判定）：两步：

1. 把 G 转成乔姆斯基范式（Chomsky normal form, CNF）：规则只有 $A \to BC$ 与 $A \to a$（另允许 $S \to \varepsilon$）；
2. 枚举所有长度 $2|w| - 1$ 的推导，看有没有生成 w 的。

引理（CNF 的推导长度）：若 G 是 CNF 且 $w \in L(G)$，则 w 的每个推导都恰好有 $2|w| - 1$ 步。理由：每个 $A \to BC$ 增加一个变量（最终要变成 $|w|$ 个终结符，需要 $|w| - 1$ 次二元展开），每个 $A \to a$ 产出一个终结符（$|w|$ 次）。所以总共 $2|w| - 1$ 步。

这条引理的意义：它把"无穷多的推导"压成"有限多"，于是枚举可行。**没有 CNF，A_{CFG} 的判定器就写不出来**。

定理（E_{CFG} 可判定）：反过来标记——先标记所有终结符，然后反复标记"规则右部全被标记"的变量，最后看起始变量是否被标记。

> $M_{E\text{-CFG}}$ = "On input $\langle G \rangle$：1. 标记 G 中所有终结符的出现；2. 重复直到没有新变量被标记：若 $A \to B_1 \cdots B_k$ 且所有 B_i 已标记，则标记 A；3. 若起始变量未被标记则接受（语言为空），否则拒绝。"

两个"标记"方向必须分清：E_{DFA} 从**起点**正向标记状态；E_{CFG} 从**终结符**反向标记变量。前者问"能否到达接受态"，后者问"能否从起始变量推出终结符串"。

定理（A_{TM} 可识别但不可判定）：识别器

> U = "On input $\langle M, w \rangle$：1. 在 w 上模拟 M；2. 若 M 接受则接受；若 M 拒绝则拒绝；若 M 不停机则本机也不停机。"

L7 把 U 称为"图灵的原初通用计算机"（universal computing machine），并引了一句历史评注：Von Neumann 说 U 启发了存储程序计算机的概念。这段话在课上是一个"理论有工程后果"的例证，但注意它是**评注**而非定理。

9.4 例子

例（EQ_{CFG} 为什么难）：DFA 的等价性靠"乘积 + 空性"，因为 DFA 是**识别器**且对布尔运算封闭。CFG 是**生成器**，两个文法生成同一个语言这件事没有有限的"局部"判据。L7 的幻灯片把这一点归因于 "(a) Because

CFGs are generators and DFAs are recognizers" 与 "(c) Because CFLs are not closed under complementation and intersection"——两个选项其实都对，讲义把它作为讨论题。

9.5 常见误区

误区	为什么错	正确做法
认为 A_{CFG} 要枚举**所有**推导	推导可以任意长，无穷多	先转 CNF，把长度锁死为 $2\|w\|-1$
把 E_{DFA} 的可达性做成"从接受态反向"	逻辑上可以，但必须与"起点"配合；混用会漏判	正向从起点，或反向从接受态，二选一并说明
用对称差证 EQ_{CFG}	CFL 不封闭于补/交	该技巧只对 DFA 有效

第 10 章 对角线法：不可判定性的第一个证明

> **对应材料**：Lec 8（Sipser §4.2）｜关键词：diagonalization, countability, one-to-one correspondence

10.1 问题：怎么"证明不存在"

要证 A_{TM} 不可判定，需要排除**每一台** TM。可用的唯一结构性工具是**计数**：如果问题比机器多，就一定有问题无法被任何机器解决。

L8 的路线是三步：

1. 建立"集合大小"的精确定义（双射）；
2. 证明 $\mathbb{R}$ 不可数、语言集合不可数；
3. 证明 TM 集合可数，于是"存在不可判定的语言"；再把某个具体语言（A_{TM}）钉死。

10.2 形式定义

定义（同大小）：集合 A 与 B **大小相同**，若存在双射（one-to-one correspondence）$f: A \to B$，即 f 同时是单射（injective）与满射（surjective）。

定义（可数）：A 可数（countable），若 A 有限或 $|A| = |\mathbb{N}|$。

定义（不可数）：不可数的集合即非可数集合。

10.3 定理与证明思路

定理（$\mathbb{Z}$、$\mathbb{Q}$ 可数）：$\mathbb{Z}$ 用"偶数位放正数、奇数位放负数"排；$\mathbb{Q}^+$ 用**对角线枚举**（把 i/j 排成二维表格，沿反对角线蛇形走），跳过非最简分数。

定理（$\mathbb{R}$ 不可数，Cantor）：假设 $\mathbb{R}$ 可数，则存在双射 $f:\mathbb{N}\to\mathbb{R}$，把 $f(i)$ 写成无穷小数。构造 x 使 x 的第 i 位小数**不等于** $f(i)$ 的第 i 位。则 x 与每个 $f(i)$ 都不同，故 x 不在列表中，与满射矛盾。

技术细节（讲义未展开但考试会踩）：小数表示不唯一（$0.4999\ldots=0.5000\ldots$），所以"不等于"要选一个避开两种表示的规则（如只取 1 和 2）。Sipser 书里有这段处理，L8 的板书略过。

推论 1（语言集合不可数）：所有语言的集合 $\mathcal{L}$ 与 $\mathbb{R}$ 等势，故不可数。

推论 2（存在不可判定的语言）：所有 TM 的集合 $\mathcal{M}$ 可数（因为每台 TM 都有有限描述，$\mathcal{M}\subseteq\Sigma^*$，而 Σ^* 可数）。$\mathcal{L}$ 不可数，故 $\mathcal{L}\setminus\mathcal{M}\neq\varnothing$——存在语言不是任何 TM 的语言。

这是全课第一个"存在性"的不可判定性证明，但它不给出具体语言。下一步才是构造。

定理（A_{TM} 不可判定）：假设 TM H 判定 A_{TM}，即

$$H(\langle M,w\rangle)=\begin{cases}\textbf{accept} & M\text{ 接受 }w\\ \textbf{reject} & \text{否则}\end{cases}$$

构造 D = "On input $\langle M\rangle$：1. 在 $\langle M,\langle M\rangle\rangle$ 上跑 H；2. 若 H 接受则拒绝，若 H 拒绝则接受。"

于是 D 接受 $\langle M\rangle$ 当且仅当 M 不接受 $\langle M\rangle$。取 $M=D$：

$$D\text{ 接受 }\langle D\rangle\iff D\text{ 不接受 }\langle D\rangle$$

矛盾。故 H 不存在。

为什么这叫对角线：把"所有 TM"排成行、把"所有串"排成列，格子 (i,j) 填"M_i 是否接受 s_j"。D 的行为定义为"在第 i 行第 i 列取反"，于是 D 这一行与每一行都至少差一格——它不可能出现在表里。这与 Cantor 构造"第 i 位取反"的小数是同一个动作。

定理（A_{TM} 不可识别）：需要一条引理。

引理：若 L 与 $\overline{L}$ 都可识别，则 L 可判定。

证明：设 M_1 识别 L、M_2 识别 $\overline{L}$。构造判定器 M = "On input w：**并行**跑 M_1 与 M_2（交替各走一步），直到其中一个接受；若 M_1 接受则接受，若 M_2 接受则拒绝。"

并行是本质的：串行跑 M_1 会在 $w\notin L$ 时卡死。这里又一次用到 dovetailing。

推论（A_{TM} 不可识别）：A_{TM} 可识别（第 9 章）但不可判定，由引理的逆否命题得 $\overline{A_{\mathrm{TM}}}$ 不可识别。而 A_{TM} 本身如果不是可识别的，会与"A_{TM} 可识别"矛盾——准确地说：**A_{TM} 可识别但不可判定，因此它的补不可识别**。L8 的幻灯片写的是 "A_{TM} is T-unrecognizable"，指的是 A_{TM} 的**补**（讲义记号在这一点上一直用 $\overline{A_{\mathrm{TM}}}$ 表示，排版丢失后容易误读，需按上下文判断）。

10.4 例子

例（对角线的三个化身）：同一套骨架在不同场景里换了"格子内容"——Cantor 的行是自然数、列是小数位、对角线对象是反序小数；停机问题的行是 TM、列是串、对角线对象是 D；层次定理（第 19 章）的行是 TM、列是串、对角线对象是那个限空间模拟的机器 B。

10.5 常见误区

误区	为什么错	正确做法
认为"不可数"就推出"不可判定"	不可数是集合论事实，不可判定是关于机器的	中间必须过一步"TM 集合可数"
把 D 写成"D 接受 $\langle D \rangle$ 则接受"	那样不产生矛盾	必须**取反**
串行跑 M_1, M_2 来判定	会在一个方向上卡死	并行/dovetailing

第 11 章 归约：把已知的难嫁接到新的问题上

> **对应材料**：Lec 9（Sipser §5.1） | 关键词：reducibility, mapping reduction, computable function

11.1 问题：只有一个不可判定问题，怎么长出更多

到第 10 章为止，我们只有一个不可判定的语言（A_{TM}）和一个不可识别的语言（$\overline{A_{\mathrm{TM}}}$）。真实世界的不可判定问题（PCP、$ALL_{\mathrm{CFG}}$、希尔伯特第十问题）与图灵机毫无字面关系。

归约（reducibility）是**传播不可判定性**的机制：如果 A 不可判定，且 A 可以"翻译"成 B，则 B 也不可判定。

L9 给了三个日常类比：矩形面积归约到边长测量；A_{NFA} 归约到 A_{DFA}（第 9 章的转换）；Pset 2 里的 PUSHER 归约到 E_{CFG}。

11.2 形式定义

定义（可计算函数, computable function）：$f : \Sigma^* \to \Sigma^*$ 可计算，若存在 TM M，对每个 w 都在带上留下 $f(w)$ 并停机。

定义（映射可归约, mapping reducibility）：$A \leq_m B$，若存在可计算函数 f 使

$$w \in A \iff f(w) \in B \quad \text{对所有 } w$$

f 称为归约（reduction）。图示是：A 的问题实例经 f 变成 B 的实例，答案保持一致。

定义（一般归约 / 图灵归约）：A 归约到 B，若可以用 B 的求解器（solver）作为黑箱来判定 A。课程用"reducible"不加修饰时通常指这种一般意义。

两者的差别（L9 强调）：

维度	一般归约	映射归约
形态	算法调用 B 的判定器	一个函数 f 变换实例
强度	更宽松（包含更多情形）	更严格（是特例）
用途	证**不可判定性**	证**不可识别性**（因为要处理补集）
反例	A_{TM} 一般归约到自己（显然），但 $A_{\text{TM}} \not\leq_m A_{\text{TM}}$ 的补方向不成立	—

L9 的 check-in 9.3 问"一般归约下，$A \leq B$ 且 B 可识别是否推出 A 可识别"，答案是 No——这正是映射归约更强的原因：它额外要求 f 把**补集也对应过去**。若 $A \leq_m B$ 且 B 可识别，则 A 可识别（直接用 f + B 的识别器）；若只有一般归约，A 的识别器可能要在"B 说不"时做出 B 自己都做不到的判断。

11.3 定理与证明思路

定理（映射归约的传递性）：若 $A \leq_m B$ 且 $B \leq_m C$，则 $A \leq_m C$（复合两个可计算函数）。

定理（向下封闭）：若 $A \leq_m B$ 且 B 可判定，则 A 可判定。

证明：M_A = "On input w：1. 算 $f(w)$；2. 在 $f(w)$ 上跑 B 的判定器；3. 输出同样结果。"

推论（逆否，最常用的形式）：若 $A \leq_m B$ 且 A 不可判定，则 B 不可判定。

定理（可识别性也向下封闭）：若 $A \leq_m B$ 且 B 可识别，则 A 可识别。

推论：若 $A \leq_m B$ 且 A 不可识别，则 B 不可识别。

定理（$HALT_{\text{TM}}$ 不可判定）：

$$HALT_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ 在 } w \text{ 上停机}\}$$

证明（归约 $A_{\text{TM}} \leq_m HALT_{\text{TM}}$ 的"一般归约"版本）：假设 R 判定 $HALT_{\text{TM}}$，构造判定 A_{TM} 的 S：

> S = "On input $\langle M, w\rangle$：1. 用 R 测试 M 在 w 上是否停机；若不停机则拒绝；2. 若停机，则模拟 M 直到停机（由 1 保证会停）；3. 若 M 接受则接受，否则拒绝。"

L9 的幻灯片在这里插了一句关键评注："Simulate M on w until it halts (**as guaranteed by R**)"——第 2 步的合法性完全来自第 1 步的判定结果。这是全课所有"用判定器消除不确定性"的证明的模板。

定理（E_{TM} 不可判定）：

$$E_{\text{TM}} = \{\langle M\rangle \mid L(M) = \varnothing\}$$

证明思路（归约 $A_{\text{TM}} \leq_m E_{\text{TM}}$ 的一般版本）：给定 $\langle M, w\rangle$，构造

> M_1 = "On input x：1. 若 $x \neq w$ 则拒绝；2. 否则在 w 上跑 M，若 M 接受则接受。"

于是 $L(M_1) = \{w\}$ 若 M 接受 w，否则 $L(M_1) = \varnothing$。因此 M 接受 w 当且仅当 $L(M_1) \neq \varnothing$，即 $\langle M, w\rangle \in A_{\text{TM}} \iff \langle M_1\rangle \notin E_{\text{TM}}$。

注意这里的技巧：M_1 **忽略自己的输入**、只关心 w。这个"写死一个串"的构造是归约里最常用的手法，第 12 章的 ALL_{CFG} 与第 11 章的 EQ_{TM} 都靠它。

定理（EQ_{TM} 与其补都不可识别）：

$$EQ_{\text{TM}} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)\}$$

证明要点：对任意 $\langle M, w\rangle$ 定义 M_1 = "忽略输入，在 w 上跑 M，接受则接受"，则

- 映射 $f(\langle M, w\rangle) = \langle M_1, M_{\text{reject}}\rangle$ 给出 $A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$；
- 映射 $g(\langle M, w\rangle) = \langle M_1, M_{\text{accept}}\rangle$ 给出 $A_{\text{TM}} \leq_m EQ_{\text{TM}}$。

两个方向都归约到 A_{TM} 的不可识别性，于是 EQ_{TM} 与它的补都不可识别。**这是"最坏"的一类问题**：连"猜"都没法猜。

11.4 例子

例（两个模板，直接背）：L9 把不可判定性的证明压成两句话：

> **证 B 不可判定**：假设 TM R 判定 B；构造 TM S 判定 A_{TM}（已知不可判定）；矛盾。
>
> **证 B 不可识别**：给出可计算函数 f 使 $w \in A_{\text{TM}} \iff f(w) \in B$，即 $A_{\text{TM}} \leq_m B$。

例（"归约"这个词的方向陷阱）：L9 结尾专门讨论了为什么叫"reduce"：把 A 归约到 B 意味着"A 不比 B 难"。所以符号是 $A \leq_m B$——**A 在左边，且 A 更简单**。幻灯片把两种可能的语义并列："bring A's difficulty down to B"还是"bring B's difficulty up to A"，并指出记号 $\leq$ 支持前者。写作时如果搞反方向，整个证明就无效——这是本课最高频的失分点。

11.5 常见误区

误区	为什么错	正确做法
把归约方向写反（证 $B \leq_m A$ 却想证 B 不可判定）	从"简单问题归约到难问题"推不出任何不可判定性	记住：**已知不可判定的在左，待证的靶子在右**
用一般归约证不可识别	一般归约不控制补集	不可识别必须用映射归约
认为 $A \leq_m A$ 总成立	对任何 A 取恒等函数即可，成立	但 $A \leq_m \overline{A}$ 一般不成立，这是关键区别

第 12 章 计算历史方法：PCP、LBA 与 ALL_{CFG}

> **对应材料**：Lec 10（Sipser §5.2）｜关键词：computation history, Post correspondence problem, LBA

12.1 问题：当待证问题"看起来跟 TM 无关"

E_{TM} 的归约很好写，因为要构造的对象本身就是一台 TM。但 PCP（多米诺匹配）是纯字符串问题，ALL_{CFG} 是纯文法问题，希尔伯特第十问题是纯数论问题。对它们，我们需要一种**通用的、能把"M 接受 w"编码成"某个组合对象存在"**的技术。

这就是计算历史方法（computation history method）。它的核心是一句话：**让待判定对象"恰好是"某个 TM 的接受计算历史**。

12.2 形式定义

定义（格局）：TM 的格局（configuration）是三元组 $(\text{状态}, \text{头位置}, \text{带内容})$。编码成字符串时，把状态**插在**头所在的符号之前：若带内容为 uav、头在 a 上、状态为 q，则编码为 uqv。

定义（接受计算历史）：TM M 在输入 w 上的接受计算历史是一串格局 $C_1, C_2, \dots, C_{\text{accept}}$，满足 C_1 是起始格局、C_{i+1} 由 C_i 按 M 的转移合法产生、C_{accept} 含 q_{accept}。编码为 $C_1 \# C_2 \# \cdots \# C_{\text{accept}}$。

定义（线性有界自动机, LBA）：一条带的 TM，头**不能移出输入所占区域**。带长随输入长度伸缩。

$$A_{\text{LBA}} = \{\langle M, w\rangle \mid M \text{ 是 LBA 且接受 } w\}$$

定义（Post 对应问题, PCP）：给定多米诺集合

$$P = \left\{ \begin{bmatrix} t_1 \\ b_1 \end{bmatrix}, \dots, \begin{bmatrix} t_k \\ b_k \end{bmatrix} \right\}$$

一个**匹配**（match）是有限序列（可重复取）使上串连接等于下串连接：$t_{i_1} \cdots t_{i_\ell} = b_{i_1} \cdots b_{i_\ell}$。

$$PCP = \{\langle P\rangle \mid P \text{ 有匹配}\}$$

12.3 定理与证明思路

定理（A_{LBA} 可判定）：

关键观察：长度为 n 的输入上，LBA 的格局数不超过 $|Q| \times n \times |\Gamma|^n$（状态 × 头位置 × 带内容）。若运行超过这么多步，必然重复格局，于是**进入循环、永不停机**。

判定器：$M_{A\text{-LBA}}$ = "On input $\langle M, w\rangle$：1. 令 $n = |w|$；2. 让 M 在 w 上跑 $|Q| \times n \times |\Gamma|^n$ 步；3. 若已接受则接受；4. 若已拒绝或仍在运行（必然在循环）则拒绝。"

这一步的技术要点：有限状态空间 + 确定性 ⇒ 超过格局数必循环。这条论证只对**空间有界**的模型成立，对一般 TM 不成立（带可以无限增长）。这是"空间有界 = 可判定"的第一例，第 17 章会把它推广成"PSPACE ⊆ 可判定"。

定理（E_{LBA} 不可判定）：

$$E_{\text{LBA}} = \{\langle M\rangle \mid M \text{ 是 } \mathbf{LBA} \text{ 且 } L(M) = \varnothing\}$$

证明思路（归约 $A_{\text{TM}} \le_m E_{\text{LBA}}$）：给定 $\langle M, w\rangle$，构造 LBA $B_{M,w}$：

> $B_{M,w}$ = "On input x：1. 检查 x 是否形如 $C_1 \# C_2 \# \cdots \# C_{\text{accept}}$，且 C_1 是 M 在 w 上的起始格局；2. 检查每对相邻格局 C_i, C_{i+1} 是否合法（C_{i+1} 由 C_i 一步得到）；3. 检查最后一个格局是否含 q_{accept}；4. 全通过则接受，否则拒绝。"

$B_{M,w}$ 只在自己的输入带上工作，且输入的每个字符只被局部读取一次——它确实是 LBA。于是 $L(B_{M,w}) \neq \varnothing$ 当且仅当 M 接受 w 有接受计算历史，即 M 接受 w。

定理（PCP 不可判定）：

L10 的路线：先定义**修改版 PCP**（MPCP，要求匹配以第一个多米诺开头），证 $A_{\text{TM}} \le_m MPCP$，再用"加星号"技巧消去这个限制。

构造直觉：多米诺分三类——起始块 $\begin{bmatrix} \# \\ \#q_0 w \end{bmatrix}$ 给出初始格局；模拟块把"状态 + 读到的符号"改写成"写入 + 新状态"，并配 $\begin{bmatrix} a \\ a \end{bmatrix}$ 让未受影响的符号原样保留；收尾块允许在到达接受态时结束。匹配的机制是**上串落后下串一个格局**：每步用转移块把下串推进一格，上串补齐对应符号。

消去限制：对 $u = u_1 \cdots u_n$ 定义 $\star u \star = *u_1 * \cdots * u_n *$ 与 $u\star = u_1 * \cdots * u_n *$，把每个块换成"上 $\star t \star$ / 下 $b\star$"与"上 $\star t$ / 下 $b\star$"两种，再加收尾块。$*$ 的位置强制匹配只能从第一块开始。

定理（ALL_{CFG} 不可判定）：

$$ALL_{\text{CFG}} = \{\langle G\rangle \mid L(G) = \Sigma^*\}$$

证明思路：构造 PDA $A_{M,w}$，使它接受**所有不是 M 在 w 上接受计算历史的串**。

> $A_{M,w}$ 的非确定分支：猜一个"错误位置"，然后 1. 起始格局不对，或 2. 某个相邻格局对不合法，或 3. 最后一个格局不含 q_{accept}。

于是 $L(A_{M,w}) = \Sigma^*$ 当且仅当**不存在**接受计算历史，即 M 不接受 w。若 ALL_{CFG} 可判定，就判定了 A_{TM} 的补——而 A_{TM} 的补不可识别，矛盾。

技术细节（L10 幻灯片提到的"反转偶数号格局"）：PDA 的栈只能后进先出，而要比较相邻两个格局需要"正着读一个、倒着读一个"。解法是把偶数号格局**反向写**，这样栈一次弹出正好能与下一个格局对齐。这是本章最精巧的一处编码，也是"为什么偏偏用 PDA 而不是 DFA"的原因。

12.4 例子

例（PCP 的小实例）：L10 的 check-in 10.1 给了一组多米诺，问是否存在匹配。这类小例子值得手推一遍，因为 PCP 的"不可判定"与"小实例很容易"并不矛盾——**不可判定性是关于一般情形的**。

问题	存在的对象
希尔伯特第十问题	整数解
A_{LBA}	被接受的串
PCP	匹配
ALL_{CFG} 的补	被拒绝的串

而那个对象**都是计算历史的某种编码**。这句话是本章的"一句话总结"，也是考试里判断"该不该用计算历史方法"的启发式。

12.5 常见误区

误区	为什么错	正确做法
认为 PCP 是"字符串匹配"所以简单	匹配允许**重复使用**多米诺且长度无界	记住可重复 + 无界是难度的来源
忘记 PCP 的匹配必须从某个位置开始	标准 PCP 不限制，MPCP 才限制	归约时先用 MPCP 再消去限制
用普通 TM 代替 LBA 做历史检验	会失去"只检验、不模拟"的语义	检验器必须在输入区域内工作

第 13 章 递归定理：让机器谈论自己

> **对应材料**：Lec 11（Sipser §6.1–6.2） | 关键词：recursion theorem, self-reference, fixed point, Gödel

13.1 问题：自指看起来是非法的

一台 TM 的描述 $\langle M \rangle$ 是一个字符串，而 M 的运行需要这个字符串作为输入。直觉上这构成循环："要运行 M 得先知道 M，要知道 M 得先构造 M。"

L11 用一个工厂类比把悖论摆出来：造汽车的工厂比汽车复杂（因为工厂要包含"造车的说明 + 机器人"），那么造工厂的工厂要比工厂更复杂——无限倒退。但生物会自我复制，所以一定有什么地方错了。

错在"复杂度必须严格递增"这个隐含假设。**自复制不需要无限复杂度，只需要"两部分 + 一份说明书"的结构。**

13.2 形式定义

定理（自复制机器, SELF）：存在 TM $SELF$，对**任意**输入，停机时带上写着 $\langle SELF\rangle$。

引理（引号引理）：存在可计算函数 $q:\Sigma^*\to\Sigma^*$，使对每个串 w，$q(w)=\langle P_w\rangle$，其中 P_w 是"打印 w 并停机"的 TM。

证明：给定 w，机器 P_w 的构造是机械的（把 w 硬编码进状态转移），故 q 可计算。

定理（递归定理, Recursion Theorem）：对任意 TM M，存在 TM R，使得对所有 w，

$$R\text{ 在 }w\text{ 上的行为}\ =\ M\text{ 在 }\langle R\rangle, w\text{ 上的行为}$$

直观说法：**任何 TM 都可以"知道自己的描述"**。

13.3 定理与证明思路

$SELF$ 的证明（两部分结构）：$SELF=AB$，其中

- $A=P_{\langle B\rangle}$：打印 B 的描述；
- B = "1. 读带上的 $\langle B\rangle$；2. 用引理算 $q(\langle B\rangle)=\langle A\rangle$；3. 拼出 $\langle AB\rangle=\langle SELF\rangle$；4. 停机。"

关键点：B 里**没有** A 的描述，只有"算出 A 的描述"这个动作。所以不循环。L11 的原话是 "1 = $P_{\langle B\rangle}$? NO, would be circular reasoning."

英文版的可执行类比（L11 的 check-in 11.1，也是全课最好的教学例子之一）：

> 写两遍下面这句话，第二次加引号："写两遍下面这句话，第二次加引号。"

第一句是模板（template），第二句是动作（action）。拼起来就得到了自己的完整描述。这个例子说明"自复制"的本质是**把模板当数据处理**——这正是 stored-program 的思想。

递归定理的证明（三部分结构）：R 由 A,B,M 三块组成：

- $A=P_{\langle BM\rangle}$；
- B = "1. 读带上 w 之后的 $\langle BM\rangle$；2. 算 $q(\langle BM\rangle)=\langle A\rangle$；3. 拼出 $\langle ABM\rangle=\langle R\rangle$；4. 把控制权交给 M，输入为 $w,\langle R\rangle$。"

于是 R 在 w 上做的事就是 M 在 $\langle R\rangle, w$ 上做的事。

使用约定（L11 强调的"moral"）：从此以后，写 TM 时可以直接写 "obtain own description $\langle M\rangle$"，就像编程语言里的 `__FILE__` 或反射（reflection）。这不是作弊，因为递归定理保证了它可被展开成合法的 TM。

定理（A_{TM} 不可判定的新证明）：假设 H 判定 A_{TM}，构造

> B = "On input w：1. 取得自己的描述 $\langle B\rangle$；2. 用 H 判断 B 是否接受 w；3. 做与 H 的答案**相反**的事。"

则 B 接受 w 当且仅当 B 不接受 w，矛盾。

与第 10 章的对角线证明对比：第 10 章的 D 需要"把 M 的描述喂给 M"这个外部动作；递归定理版把这个动作内化了。**两个证明是同一个矛盾，只是一个用外部编码、一个用内部自指。**

定理（不动点定理, fixed-point theorem）：对任意可计算函数 $t : \Sigma^* \to \Sigma^*$，存在 TM F 使 $L(F) = L(G)$，其中 $\langle G \rangle = t(\langle F \rangle)$。

读法：把 t 看成一个"程序变换器"（优化器、混淆器），则总存在一个程序，被变换后**行为不变**。证明：构造

> F = "On input w：1. 取得 $\langle F \rangle$；2. 算 $t(\langle F \rangle) = \langle G \rangle$；3. 在 w 上模拟 G。"

于是 F 与 G 行为相同，而 $\langle G \rangle = t(\langle F \rangle)$。

这个定理的哲学含义：任何"程序改写工具"都存在无法改变其语义的输入。对编译器优化、程序混淆、病毒变形都有直接推论。

定理（MIN_{TM} 不可识别）：

$$MIN_{\mathrm{TM}} = \{\langle M \rangle \mid M \text{ 是最小 } \mathrm{TM}\}$$

其中"最小"指"不存在更短的等价 TM"。证明：假设枚举器 E 枚举 MIN_{TM}，构造

> C = "On input w：1. 取得 $\langle C \rangle$；2. 跑 E 直到出现某台 TM D 满足 $|\langle D \rangle| > |\langle C \rangle|$；3. 在 w 上模拟 D。"

则 $L(C) = L(D)$ 且 C 比 D 短，故 D 不最小，与 $D \in MIN_{\mathrm{TM}}$ 矛盾。

注意这个证明的结构：它把"枚举 + 比较长度"变成了"构造一个比任何被枚举者都短的机器"，是"对角线的定量版"。

13.4 例子

例（Gödel 不完备定理的骨架）：设 T 是"合理"的形式系统，要求两条性质：**可靠性**（有证明的命题为真）与**可检验性**（$\{\langle P, S \rangle \mid P \text{ 是 } S \text{ 的证明}\}$ 可判定）。由可检验性，可证命题的集合是**可识别的**；而 $\overline{A_{\mathrm{TM}}}$ 不可识别，故存在真的 $\langle M, w \rangle \notin A_{\mathrm{TM}}$ 无法被证明。

L11 用递归定理造出具体句子：令 S 为"$\langle R, 0 \rangle \in A_{\mathrm{TM}}$"，其中

> R = "On any input：1. 取得 $\langle R \rangle$，由此得到 S；2. 对每个可能的证明 $P = P_1, P_2, \ldots$：若 P 证明 S，则接受；否则继续。"

结论：(1) S 无证明；(2) S 为真。**关键在 (2)**：若 S 假，则 R 不接受 0（没找到证明）；但"S 假"本身可证，于是 R 会找到证明并接受，导致 S 真——矛盾。整个构造的每一步都对应一个 TM 动作，没有"人类直觉"的跳跃。

13.5 常见误区

误区	为什么错	正确做法
认为自指必须"先有完整的自己"	两部分结构（模板 + 动作）即可	记住 $A = P_{\langle B \rangle}$，$B$ 里只有"算 q"的动作
在 B 里直接写 A 的描述	循环定义，非法	用引理 q 从 $\langle B \rangle$ 算出 $\langle A \rangle$
把递归定理当"允许作弊"	它是有证明的定理，可展开	可以放心用 "obtain own description"，但要能说明展开方式

第 14 章 时间复杂性：从"能不能算"到"算得多快"

> **对应材料**：Lec 12（Sipser §7.1） | 关键词：TIME(t(n)), P, polynomial related, PATH

14.1 问题：复杂度理论的第一道坎是"模型依赖"

第 13 章之前的所有结论都是**模型无关**的（丘奇–图灵论题保证了这一点）。一旦开始数步数，模型立刻变得重要：

模型	判定 $\{a^n b^n \mid n \geq 0\}$ 的步数
单带 TM	$O(n^2)$，改进后 $O(n \log n)$
多带 TM	$O(n)$

L12 用这张表把"model dependence"摆到台面上，然后立刻给出解决办法：**只关心"多项式等价"这个粗粒度**。

14.2 形式定义

定义（运行时间）：TM M 在时间 $t(n)$ 内运行，若对所有长度 n 的输入，M 至多走 $t(n)$ 步后停机。这是**最坏情况**（worst-case）界。

定义（时间类）：

$$TIME(t(n)) = \{L \mid \text{某台确定性单带 TM 在 } O(t(n)) \text{ 时间内判定 } L\}$$

定义（多项式相关, polynomially related）：两个模型多项式相关，若互相模拟的开销都是多项式。L12 列的清单：单带 TM、多带 TM、多维 TM、RAM（随机访问机）、元胞自动机——**全部多项式相关**。

定义（类 P）：

$$P = \bigcup_{k \geq 0} TIME(n^k)$$

14.3 定理与证明思路

定理（单带模拟多带有多项式开销）：若多带 TM 在 $t(n)$ 内判定 L，则 $L \in TIME(t^2(n))$。

证明：单带模拟一步要扫描全带（长度 $O(t(n))$），共 $t(n)$ 步，总计 $O(t^2(n))$。

推论：P 的定义不依赖"单带还是多带"——这是 P 成为"现实可解"的合理形式化的前提。

定理（$\{a^n b^n\}$ 的单带下界）：单带 TM 判定它需要 $\Omega(n \log n)$ 步。L12 明确说 "You are not responsible for knowing the proof"，但**结论值得记住**：它说明单带模型的时间层次是真的有结构的，且在这门课里是第 19 章之前唯一一个具体下界。

定理（$PATH \in P$）：

$$PATH = \{\langle G, s, t\rangle \mid G \text{ 是有向图，有从 } s \text{ 到 } t \text{ 的路径}\}$$

判定器：从 s 出发反复标记邻居，直到没有新节点被标记，看 t 是否被标记。

复杂度核算（这是 L12 的重点，因为它是第一个完整的时间分析）：

步骤	次数	每次代价	小计
标记 s	1	$O(1)$	$O(1)$
外层循环	$\leq n$ 轮	—	—
内层遍历已标记节点	$\leq n$	—	—
扫描 G 的边	1	$O(n^2)$	$O(n^3)$

总计 $O(n^3)$，多项式。L12 的幻灯片把"To show polynomial time: Each stage should be clearly polynomial and the total number of steps polynomial"写成一条方法论——**分段核算，不要试图写出精确公式**。

定理（$HAMPATH$ 的暴力算法是指数级）：

$$HAMPATH = \{\langle G, s, t\rangle \mid G \text{ 有从 } s \text{ 到 } t \text{ 的哈密顿路径}\}$$

暴力枚举所有长度 n 的路径：路径数可达 $n!$，故算法是 $n!$ 级。L12 的 check-in 12.3 问"$HAMPATH$ 是否在 P"，正确选项是 (d) Probably No——注意用词：**没人在 2020 年能证明它不在 P**，这正是第 15 章 P vs NP 的入口。

14.4 例子

例（两个 check-in 的用意）：12.1 问"$O(n^2)$ 还能改进吗"，答"$O(n \log n)$ 可以、$O(n)$ 不行"——这说明固定模型下"最优"有精确含义；12.3 问"$HAMPATH$ 是否在 P"，答"(d) Probably No"——注意用词，**至今没人能证明它不在 P**。

14.5 常见误区

误区	为什么错	正确做法
认为 P 依赖具体模型	所有合理模型多项式相关	P 是模型无关的类
把"多项式时间"当成"实际很快"	n^{100} 也是多项式	P 是"可解性"的粗分界，不是工程判据
用 Ω 下界去证"不在 P"	下界必须对**所有**算法成立	具体算法的指数复杂度不构成下界

第 15 章 NP：用"证书"重述非确定性

> **对应材料**：Lec 14（Sipser §7.2–7.3）｜关键词：NTIME, NP, certificate, polynomial-time reduction

15.1 问题：非确定性怎么变成"可验证"

第 2 章的非确定性是"猜测"，第 8 章证明了 NTM 与 TM 等价（能力上）。但在复杂性层面，NTM 的"猜"有一个非常有用的解释：**猜出来的东西就是证书（certificate）**。

L14 的对照句是全章的核心：

类	一句话
P	能**快速判定**成员资格的语言
NP	能**快速验证**成员资格的语言

15.2 形式定义

定义（NTM 的运行时间）：NTM 判定器在时间 $t(n)$ 内运行，若**所有分支**都在 $t(n)$ 步内停机。

注意"所有分支"——这是与"存在接受分支"不同的要求，它保证了 NTM 一定给出答案。

定义（非确定性时间类）：

$$NTIME\big(t(n)\big) = \{L \mid \text{某台单带 NTM 在 } O(t(n)) \text{ 时间内判定 } L\}, \qquad NP = \bigcup_k NTIME(n^k)$$

定义（多项式时间归约）：$A \le_P B$，若存在**多项式时间可计算**的 f 使 $w \in A \iff f(w) \in B$。

定义（证书视角的 NP，等价刻画）：$L \in NP$ 当且仅当存在多项式 p 与多项式时间谓词 V，使

$$w \in L \iff \exists c \, (|c| \le p(|w|) \ \wedge \ V(w, c) = 1)$$

c 称为证书（certificate，也叫 witness）。这个刻画不在讲义的形式定义里，但 L14 的幻灯片用"Guess bits of v_i / Check"的图表达了同一件事，且在第 16 章的 Cook–Levin 证明里被直接使用。**标注：这是标准等价**

刻画，属课外补充但与本课内容一致。

15.3 定理与证明思路

定理（$HAMPATH \in NP$）：NTM = "On input $\langle G, s, t\rangle$（G 有 n 个节点）：1. 非确定性地写下 n 个节点 $v_1, \ldots, v_n$；2. 接受若 $v_1 = s$、$v_n = t$、每个 (v_i, v_{i+1}) 是边、且无重复。"

运行时间是 $O(n)$ 级（写 n 个节点 + 检查 n 条边），多项式。

这里的"猜"就是"猜一条路径"，而验证只需线性时间。这就是"快速验证"的具体含义。

定理（$COMPOSITES \in NP$）：

$$COMPOSITES = \{x \mid x \text{ 是合数，用二进制写}\}$$

NTM：猜一个因子 d（$1 < d < x$），检查 $d \mid x$。

编码的陷阱（L14 特意强调）：若用**一元**（unary）编码，x 写成 1^x，长度是 x，猜因子只需 $O(\log x)$ 位——但输入长度已经是 x，所以"多项式时间"变得毫无意义（任何 x 的多项式都远大于 $\log x$）。L14 的原话："Bad encoding: write number 3 in unary: $1 = 111 \cdots 1$, exponentially longer."

这条观察的重要性：复杂性类的定义**依赖编码**。标准约定是用二进制（或任何"紧凑"编码）。这是本课唯一一处明确讨论编码影响的地方，考试里常以"为什么用二进制"的形式出现。

L14 还提了一句："Theorem (2002): $COMPOSITES \in P$"（AKS 素性测试），并说明不证。这条结论把 $COMPOSITES$ 从"NP 的典型例子"变成了"其实是 P 的"，所以现在的教材更常用 $HAMPATH$ 或 SAT 做 NP 的例子。

定理（$A_{\text{CFG}} \in P$）：用动态规划（dynamic programming, DP）。朴素递归 $R(G, w, A)$ 对每个切分 $w = w_1 w_2$ 与每条规则 $A \to BC$ 递归测试两个子串，每层 $O(n)$ 个调用、深度 $O(\log n)$，总调用指数级。但子问题只有 $O(n^2)$ 个（每个子串一个），记下来即可：按子串长度递增填表，对每个子串尝试所有切分与规则，总时间 $O(n^3)$。这就是 CYK 算法（Cocke–Younger–Kasami），课程未用这个名字但结构一致。

推论（每个 CFL 都在 P 里）：L14 的 check-in 14.2 问这个，答案 Yes。这条推论把第 9 章的"A_{CFG} 可判定"加强成了"$A_{\text{CFG}} \in P$"。

定理（SAT 的地位，Cook–Levin 1971）：$SAT \in P \Rightarrow P = NP$。证明见第 16 章。

定理（$\leq_P$ 的向下封闭）：若 $A \leq_P B$ 且 $B \in P$，则 $A \in P$。

15.4 例子

例（$P \subseteq NP$）：确定性 TM 是非确定性 TM 的特例，故 $P \subseteq NP$。L14 的 slides 画了两个包含的椭圆。

15.5 常见误区

误区	为什么错	正确做法
把 NP 理解成"非多项式"	NP = nondeterministic **polynomial**	记住是"非确定性多项式时间"
认为 NP 里的问题必须"猜"	有确定性的验证器刻画即可	用证书定义往往更好写
认为 $P \neq NP$ 已被证明	至今开放	所有"NP 完全问题没有多项式算法"都是**猜想**

第 16 章 NP 完全性与 Cook–Levin 定理

> **对应材料**：Lec 15–16（Sipser §7.4–7.5）｜关键词：NP-complete, Cook–Levin, tableau, 3SAT, CLIQUE

16.1 问题：怎么知道"这个 NP 问题是最难的"

NP 里有无穷多问题。要在其中找"最难"的，需要一个**可操作的归约起点**。Cook–Levin 定理提供的就是这个起点：SAT 是 NP 完全的，即**每一个** NP 问题都能多项式时间归约到它。

一旦有了 SAT，证明其他问题 X 是 NP 完全的就变成两步：$X \in NP$，且 $SAT \leq_P X$（或用任何已知 NP 完全问题做起点）。

16.2 形式定义

定义（NP 完全, NP-complete）：L 是 NP 完全的，若

1. $L \in NP$；
2. 对**所有** $A \in NP$，$A \leq_P L$。

定义（NP 难, NP-hard）：只要求第 2 条，不要求 $L \in NP$。

定义（CNF / 3CNF）：

术语	定义
文字（literal）	变量或其否定
子句（clause）	若干文字的析取（OR）
CNF	若干子句的合取（AND）
3CNF	每个子句恰有 3 个文字的 CNF

$$SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的布尔公式}\}, \qquad 3SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 } 3\text{CNF}\}$$

定义（团、团问题）：k–团是 k 个两两相邻的节点。

$$CLIQUE = \{\langle G, k\rangle \mid G \text{ 含 } k\text{-团}\}$$

16.3 定理与证明思路

定理（SAT 是 NP 完全的，Cook–Levin）：$SAT \in NP$ 显然（猜赋值，验证）。难点是第 2 条。

证明思路（tableau 方法）：设 $A \in NP$ 由 NTM N 在时间 n^k 内判定。构造多项式时间归约 $f : w \mapsto \phi_{N,w}$，使

$$w \in A \iff \phi_{N,w} \text{ 可满足}$$

$\phi_{N,w}$ 的语义是"**存在一个 N 在 w 上的接受 tableau**"。tableau 是一张 $n^k \times n^k$ 的表，第 i 行是第 i 步的格局。

$$\phi_{N,w} = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$$

子公式	含义	规模
ϕ_{cell}	每个格子恰有一个符号（用 $O(1)$ 个子句/格）	$O(n^{2k})$
ϕ_{start}	第一行是 $q_0 w_1 \cdots w_n \sqcup \cdots$	$O(n^k)$
ϕ_{move}	每个 2×3 窗口是**合法**的（与 δ 一致）	$O(n^{2k})$
ϕ_{accept}	某一行含 q_{accept}	$O(n^{2k})$

ϕ_{move} 为什么是 2×3 窗口：一格的变化只取决于它自己和左右邻居——共 3 格；再考虑上下两行就是 $2 \times 3 = 6$ 格。合法性是"局部"的，所以可以用常数大小的子句枚举所有非法窗口并禁止它们。

关键引理：若 tableau 中每个 2×3 窗口都合法，则整个 tableau 对应一条合法的计算路径。证明对行数归纳：第 1 行是起始格局；每一步的局部合法性保证下一行是上一行的合法后继。

复杂度：$\phi_{N,w}$ 的大小是 $O(n^{2k})$，故 f 是多项式时间可计算。

定理（$3SAT$ 是 NP 完全的）：$SAT \le_P 3SAT$。给每个子句做"拆分"：

- 子句有 3 个文字：保留；
- 子句有 1–2 个文字：用新变量补齐（如 $(a \vee b) \mapsto (a \vee b \vee z) \wedge (a \vee b \vee \bar{z})$）；
- 子句有 $k > 3$ 个文字：用 $k - 3$ 个新变量链式拆分。

L16 幻灯片给了一个具体的等价变换例子：$a \wedge b \to c$ 的 4 个子句展开。注意**这里不保持逻辑等价，只保持可满足性**——L16 原文明确写 "(Note: ϕ and ϕ' are not logically equivalent)"。这是 $3SAT$ 归约最容易被误解的一点。

定理（$3SAT \le_P CLIQUE$）：给定 3CNF ϕ 有 k 个子句，构造图 G：每个文字一个节点（共 $3k$ 个）；同一子句内**不相连**；不同子句的节点相连，**除非**互补。结论：ϕ 可满足 $\iff$ G 有 k–团。

双向：(→) 每子句取一个真文字，k 个两两不冲突且有边；(←) 任何 k-团必每子句取一个、不含互补对，于是赋值使每子句为真。L15 的 check-in 15.1 问"是否依赖每子句恰 3 个文字"，答案 **(c) No**——该归约实际给出 $CNF \leq_P CLIQUE$。

定理（$3SAT \leq_P HAMPATH$）：用"变量 gadget + 子句 gadget + zig-zag"模拟赋值，细节在 Sipser §7.5。L15 的 check-in 15.3 问"改成无向边是否还成立"，答案 **No**——构造依赖有向 zig-zag 强制路径方向，**gadget 的方向性常是构造的关键**。

定理（$CLIQUE \leq_P VERTEX\text{-}COVER$，教材内容）：$G$ 有 k-团 $\iff$ 补图 $\overline{G}$ 有 $|V| - k$ 的顶点覆盖。L15 未讲，属 §7.5。

16.4 例子

例（NP 完全性的"证据"含义）：L15 的幻灯片列出两条重要性：(1) 计算不可行性的证据；(2) 证明 $P \neq NP$ 的候选靶子。注意用词是"evidence"不是"proof"——**NP 完全性是"没有多项式算法"的强证据，不是定理**。

16.5 常见误区

误区	为什么错	正确做法
归约方向反了（证 $X \leq_P 3SAT$）	那只说明 X 不比 $3SAT$ 难	必须 $3SAT \leq_P X$
忘记证 $X \in NP$	NP 完全要求两条	先给证书，再给归约
认为 $3SAT \leq_P CLIQUE$ 要求恰好 3 个文字	构造对任意子句长度都成立	记住答案 (c)

第 17 章 空间复杂性：PSPACE 与 Savitch 定理

> **对应材料**：Lec 17–18（Sipser §8.1–8.3） | 关键词：SPACE(f), NSPACE(f), Savitch, TQBF, PSPACE-complete

17.1 问题：空间比时间"便宜"在哪

时间不可回收，空间可以复用。这个不对称导致两个后果：**空间类的层级更细**（$NL \subseteq SPACE(\log^2 n) \subsetneq PSPACE$），且**非确定性能被确定性地模拟而只平方空间**（Savitch）。这是复杂性理论里少数几个"非确定性几乎免费"的场景。

17.2 形式定义

定义（空间复杂度）：TM M 在空间 $s(n)$ 内运行，若对所有长度 n 的输入，M 停机且最多用 $s(n)$ 个带格。

$$SPACE(s(n)) = \{L \mid \text{某确定性单带 TM 在 } O(s(n)) \text{ 空间内判定 } L\}$$

$NSPACE$ 同理（要求**所有分支**都在该空间内停机）。

$$PSPACE = \bigcup_k SPACE(n^k), \qquad NPSPACE = \bigcup_k NSPACE(n^k)$$

定义（PSPACE 完全）：L 是 PSPACE 完全的，若 $L \in PSPACE$ 且对所有 $A \in PSPACE$ 有 $A \leq_P L$。

注意：定义里用 $\leq_P$ 而不是 $\leq_{\log}$。L18 的 check-in 18.1 专门解释了为什么：**归约必须比类本身"弱"**，否则类内所有问题会互相归约、全部变成完全问题，概念就失去意义。

17.3 定理与证明思路

定理（时间与空间的两个方向）：

1. $TIME(f(n)) \subseteq SPACE(f(n))$：$f(n)$ 步最多写 $f(n)$ 格；
2. $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$：$f(n)$ 格的配置数有限（$|Q| \cdot n \cdot f(n) \cdot |\Gamma|^{f(n)}$），超过就循环。

推论：$P \subseteq PSPACE$。

定理（$NP \subseteq PSPACE$）：$SAT \in PSPACE$（穷举赋值，只需存一个赋值 + 计数器），且 $\leq_P$ 保持 PSPACE 成员资格。

定理（Savitch）：对 $f(n) \geq n$，$NSPACE(f(n)) \subseteq SPACE(f^2(n))$。

推论：$PSPACE = NPSPACE$。这是本课第一个"非确定性不增加能力"的重要类等式（对比 P vs NP 的开放）。

证明思路（可达性的分治）：核心是一个递归过程 `REACH(C1, C2, t)`："能否在 t 步内从格局 C_1 走到 C_2？"

- $t = 1$：直接查 $C_1 = C_2$ 或一步可达；
- $t > 1$：枚举所有**中间格局** C_m（用 $f(n)$ 空间），递归检查 `REACH(C1,Cm,t/2)` 与 `REACH(Cm,C2,t/2)`。

空间分析（这是 Savitch 的全部技术内容）：每层递归只存一个格局（$O(f(n))$ 空间），深度 $\log t = O(f(n))$，所以总空间 $O(f^2(n))$。**注意时间是指数级的**——这个算法用时间换空间，但 PSPACE 只关心空间。

定理（$TQBF$ 是 PSPACE 完全的）：

$$TQBF = \{\langle \phi \rangle \mid \phi \text{ 是 } \mathbf{TRUE} \text{ 的全量化布尔公式}\}$$

$TQBF \in PSPACE$：递归求值，每层只需存一个变量赋值，深度 = 量词数 $\leq n$，空间 $O(n)$。

PSPACE 难（L18 的构造）：设 $A \in PSPACE$ 由 TM M 在 n^k 空间内判定，要造 QBF $\phi_{M,w}$ 表达"M 接受 w"。两次失败给出关键教训：直接用 Cook–Levin 的 tableau 是 $n^k \times 2^{O(n^k)}$；用递归 $\phi_{C_1,C_2,t} = \exists C_m(\phi_{C_1,C_m,t/2} \wedge \phi_{C_m,C_2,t/2})$ 则每层**复制**两份子公式，仍是指数。

成功的写法：用 $\forall$ 把"复制"变成"共享"：

$$\phi_{C_1,C_2,t} = \exists C_m \, \forall (C_3, C_4) \in \{(C_1, C_m), (C_m, C_2)\} \, \phi_{C_3,C_4,t/2}$$

因为 $\forall(x \in \{a,b\})\psi(x) \equiv \psi(a) \wedge \psi(b)$，语义不变，但公式里 ϕ 只出现**一次**，于是每层只增加 $O(n^k)$，总大小 $O(n^{2k})$。这也解释了为什么 QBF 恰好刻画 PSPACE：**量词交替 = 空间有界的递归**。

定理（$LADDER_{DFA} \in PSPACE$）：ladder 是一串等长串，相邻串只差一个符号。

$$LADDER_{DFA} = \{\langle B, u, v\rangle \mid L(B) \text{ 含一条从 } u \text{ 到 } v \text{ 的 ladder}\}$$

- 在 $NSPACE(n)$ 里：非确定性地猜 ladder，每步只存当前串（$O(n)$ 空间），猜 $2^{O(n)}$ 步；
- 在 $SPACE(n^2)$ 里：用 Savitch 式的分治 + 中点枚举，深度 $O(n)$、每层 $O(n)$。

17.4 例子

例（$coNP \subseteq PSPACE$）：$coNP = \{L \mid \overline{L} \in NP\}$。$PSPACE$ 对补封闭（确定性空间类总对补封闭，因为判定器可以直接翻转输出），所以 $coNP \subseteq PSPACE$。L17 的类图把 $coNP$ 与 NP 画成两个可能相交的椭圆，都在 $PSPACE$ 内。

17.5 常见误区

误区	为什么错	正确做法
把 PSPACE 完全性定义里的归约写成 $\leq_{\log}$	定义用 $\leq_P$（L18 讨论过为什么）	记 $\leq_P$，另注 $\leq_{\log}$ 也够用（第 22 章用到）
认为 Savitch 也改进了时间	它只省空间，时间指数	空间类等式，不是时间类
用 tableau 直接证 TQBF 完全性	大小指数，不是多项式归约	用 ∀ 共享子公式

第 18 章 L、NL 与 $NL = coNL$

> **对应材料**：Lec 19–20（Sipser §8.4）｜关键词：log space, log-space transducer, NL-complete, Immerman–Szelepcsényi

18.1 问题：亚线性空间怎么定义

若空间可以小于输入长度，"把输入存下来"就变得比总预算还贵。解决办法是**换模型**：用两条带的 TM，一条**只读输入带**（不计入空间），一条读写工作带（计入空间）。

L19 的原话："To define sublinear space computation, do not count input as part of space used."

18.2 形式定义

定义：

$$L = SPACE(\log n), \qquad NL = NSPACE(\log n)$$

直觉：$\log n$ 空间只够存**常数个指向输入的指针**（每个指针 $O(\log n)$ 位）。所以 L 与 NL 刻画的是"只能用常数个指针的算法"。

定义（log 空间转换器, log-space transducer）：三带 TM：只读输入带、$O(\log n)$ 工作带、**只写**输出带。它计算的函数称为 log 空间可计算。

定义（log 空间归约）：$A \leq_{\log} B$，若存在 log 空间可计算的 f 使 $w \in A \iff f(w) \in B$。

为什么输出带只写：若允许回读输出，输出长度可能超过 $\log n$，就破坏了空间约束。这个细节是定义严谨性的关键。

18.3 定理与证明思路

定理（$L \subseteq P$）：空间 $O(\log n)$ 的 TM 的格局数是

$$|Q| \times n \times O(\log n) \times |\Gamma|^{O(\log n)} = O(n^k)$$

多项式，故运行时间多项式。**注意这里用到了"空间有界 ⇒ 配置数有限 ⇒ 时间有界"这条第 12 章的思想。**

定理（$NL \subseteq SPACE(\log^2 n)$）：Savitch 定理对 $f(n) = \log n$ 直接适用（要求 $f(n) \geq \log n$，$\log n$ 满足）。

定理（$NL \subseteq P$）：用**格局图**（configuration graph）$G_{M,w}$：节点是所有格局，边是"一步可达"。

- 若 M 是 log 空间 NTM，则 $G_{M,w}$ 有 $O(n^k)$ 个节点；
- M 接受 $w \iff G_{M,w}$ 中从起始格局到接受格局有路径。

于是：构造格局图（多项式时间）→ 跑 $PATH$ 的判定器 → 得 $NL \subseteq P$。

定理（$PATH$ 是 NL 完全的）：

1. $PATH \in NL$：非确定性地猜一条路径，只存"当前节点 + 步数计数器"（$O(\log n)$）。
2. 对任意 $A \in NL$，$A \leq_{\log} PATH$：$f(w) = \langle G_{M,w}, c_{\text{start}}, c_{\text{accept}} \rangle$。**关键**：$f$ 必须在 log 空间内可算，而 $G_{M,w}$ 有 $O(n^k)$ 条边，不能整体写出。做法是**按需生成**：对每一对格局 (C_i, C_j)，现场判定"是否一步可达"，是则输出这条边。

这就是 log 空间归约的精髓：输出可以很长，但**生成过程**只需常数个指针。L20 的 check-in 20.1 问"$f(x)$ 能有多长"，答案是 (c) at most polynomial in n——输出长度多项式，但工作空间仍是对数。

定理（$2SAT$ 是 NL 完全的）：$PATH \leq_{\log} 2SAT$（实际上是 $\overline{PATH} \leq_{\log} 2SAT$，因为 2SAT 的不可满足性对应路径可达）。

构造：对图 G 的每个节点 v 设变量 x_v；每条边 (u, v) 给子句 $(x_u \to x_v)$，即 $(\bar{x}_u \vee x_v)$；再加 $(x_s \vee x_s)$ 与 $(x_t \to x_s)$。结论：G 有 $s \to t$ 路径 $\iff$ 公式不可满足。

为什么这给出 NL 完全性：$2SAT \in NL$（L20 说 "good exercise"），且 $\overline{PATH} \leq_{\log} 2SAT$，而 $\overline{PATH} \in NL$ 由下一条定理保证。

定理（Immerman–Szelepcsényi：$NL = coNL$）：NL 对补封闭。

证明结构（L20–L21）：定义 $Reach(G, s, t)$（t 从 s 可达，即 $PATH$）、$R(G, s) = \{v \mid v \text{ 从 } s \text{ 可达}\}$、$c(G, s) = |R(G, s)|$。要证 $\overline{PATH} \in NL$，只需把"数出 $|R|$"和"判定可达"互相归约：给定 c 就能校验 $Reach$（遍历节点、猜路径、数个数，若数与 c 不符则拒绝），给定 $Reach$ 就能算出 c（逐个测可达并累加）。于是 $\overline{PATH} \in NL$，得 $NL = coNL$。

核心技巧（"数数 + 校验"）：验证"t 不可达"时，机器**猜出** c，再对每个节点非确定性地"猜可达并给路径"或"跳过"，最后检查"数到的可达节点数恰为 c 且 t 不在其中"。猜错的分支失败，但**只要存在一条正确分支**即可——这就是非确定性的用法：c 是一个 $O(\log n)$ 位的证书。

18.4 例子

例（$L = NL$ 是开放问题）：与 $P = NP$ 并列的著名开放问题。已知的分离只有 $NL \subsetneq PSPACE$（由层次定理，第 19 章）。

18.5 常见误区

误区	为什么错	正确做法
把输入带算进空间	那样 $\log n$ 类就没有意义	只读输入带不计
认为 log 空间归约可以缓存输出	输出可以多项式长	按需重新计算
把 $NL \subseteq P$ 证成 $NL \subseteq L$	格局图是多项式大的	构造图 + $PATH$ 判定

第 19 章 层次定理：证明"更多资源真的能做更多事"

> **对应材料**：Lec 21–22（Sipser §9.1–9.2）｜关键词：hierarchy theorem, space constructible, diagonalization, EXPSPACE

19.1 问题：类之间是不是"真的"不同

到第 18 章为止，我们只知道一串包含关系 $L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE$，但**一个严格包含都没证**（$P \neq NP$ 是开放问题）。层次定理（hierarchy theorems）是唯一能给出严格分离的工具。

它的证明用的是第 10 章的对角线法，只是把"M 是否接受 x"换成了"M 是否在 $f(n)$ 空间内判定 x"。

19.2 形式定义

定义（空间可构造, space constructible）：函数 $f : \mathbb{N} \to \mathbb{N}$ 是空间可构造的，若存在 TM 在 $O(f(n))$ 空间内算出 $1^{f(n)}$（即把 $f(n)$ 个格子标出来）。

定义（时间可构造）：类似，要求 $O(f(n))$ 时间内算出。

定义（EXPTIME / EXPSPACE）：

$$EXPTIME = \bigcup_k TIME(2^{n^k}), \qquad EXPSPACE = \bigcup_k SPACE(2^{n^k})$$

定义（EXPSPACE 完全）：$L \in EXPSPACE$ 且所有 $A \in EXPSPACE$ 都有 $A \leq_P L$。

19.3 定理与证明思路

定理（空间层次定理）：对空间可构造的 f，

$$SPACE(o(f(n))) \subsetneq SPACE(f(n))$$

证明思路（对角线）：构造 TM B：

> B = "On input w：1. 标出 $f(|w|)$ 个格子，若用完则拒绝；2. 若 $w \neq \langle M \rangle 10^*$ 对某 TM M，拒绝；3. 在 w 上模拟 M，限制在 $f(|w|)$ 空间内；4. 若 M 接受则拒绝，若 M 拒绝则接受。"

于是 $L(B) \in SPACE(f)$，但 $L(B)$ 与任何在 $o(f)$ 空间内运行的 M 都不同。**三个技术修正**（L21 的幻灯片逐条列出）：

1. **常数因子问题**：M 可能只需 $o(f)$ 空间但常数很大，在小输入上 B 模拟不了。修正是"对无穷多输入模拟 M"。
2. **M 可能 loop**：B 必须总停机。修正是"最多模拟 2^f 步"。
3. **怎么算 f**：需要 f 空间可构造。这就是定义里那条技术条件的作用。

定理（时间层次定理）：对时间可构造的 f，

$$TIME(o(f(n)/\log f(n))) \subsetneq TIME(f(n))$$

为什么多一个 $\log f$ 的损失：B 需要数步数，计数器有 $\log f$ 位，且必须跟着读写头移动，每步多花 $O(\log f)$。所以为了在 f 步内停机，只能模拟 $f/\log f$ 步。L21 的解释是"Cost of moving it adds a $O(\log f)$ overhead factor"。

推论（一系列严格分离）：

分离	依据
$NL \subsetneq PSPACE$	$NL \subseteq SPACE(\log^2 n)$，而 $\log^2 n = o(n)$

分离	依据
$P \subsetneq EXPTIME$	时间层次定理
$PSPACE \subsetneq EXPSPACE$	空间层次定理
$NP \subsetneq NEXPTIME$	非确定性时间层次定理（教材）

推论（$EQ_{REX\uparrow} \notin PSPACE$）：L22 用带指数的正则表达式构造了一个**自然的** EXPSPACE 完全问题：

$$EQ_{REX\uparrow} = \{\langle R_1, R_2\rangle \mid R_1, R_2 \text{ 带指数记号的正则表达式且等价}\}$$

构造思路：给 $A \in EXPSPACE$（由 2^{n^k} 空间 TM M 判定），归约出两个正则表达式 R_1, R_2，使 $L(R_1)$ 是"除了 M 在 w 上的拒绝计算历史之外的所有串"，$L(R_2) = \Delta^*$。于是 $L(R_1) = L(R_2)$ 当且仅当**不存在**拒绝历史，即 M 接受 w。

这个问题的意义：它是"可证明不可行"的第一个**自然**问题——不是人为的对角线构造，而是一个关于正则表达式的、可以写在教科书习题里的问题。

定理（$EQ_{REX} \in PSPACE$）：不带指数时，等价性在 PSPACE 内：把两个正则表达式转成 NFA，非确定性地猜一个"见证串"（长度 $2^{O(n)}$），同时模拟两个 NFA，看是否在一个串上判定不同。空间只需存两个状态集。

19.4 例子

这是一道非常漂亮的题：它展示了"我们无法解决单个等式，但能证明它们不能同时成立"。

例（oracle 的教训）：L22 证明了存在 oracle A 使 $P^A = NP^A$（取 $A = TQBF$，因为 $NP^{TQBF} \subseteq NPSPACE = PSPACE \subseteq P^{TQBF}$）。由于对角线证明在 oracle 下依然成立，**任何"纯对角线"的证明都不可能分离 P 与 NP**。这个结果（Baker–Gill–Solovay 的课程版本）解释了为什么 $P \neq NP$ 这么难：它需要非相对化的技术。

19.5 常见误区

误区	为什么错	正确做法
认为层次定理能证 $P \neq NP$	它们只分离"差一个多项式以上"的类	P 与 NP 是同量级的类
忽略"可构造"条件	不构造就无法在机器里"标出 $f(n)$ 格"	引用定理时写明 f 可构造
认为时间层次定理的形式是 $o(f) \subsetneq TIME(f)$	有 $\log f$ 的损失	记住 $o(f/\log f)$

第 20 章 概率计算与 BPP

> **对应材料**：Lec 23–24（Sipser §10.2）｜关键词：probabilistic TM, BPP, branching program, arithmetization, Schwartz–Zippel

20.1 问题：允许"掷硬币"会不会变强

NP 的"猜"要求猜对；掷硬币只要求**大概率**对。BPP 定义的就是"掷硬币能在多项式时间内大概率答对"的语言。它比 P 宽，但（猜想）不比 P 强多少——这是"随机性是否可去随机化"的问题。

20.2 形式定义

定义（概率图灵机, PTM）：每步有 1 或 2 个选择的 NTM，2 个选择时各 1/2 概率。

定义（误差概率）：PTM M 以误差 ϵ 判定 L，若对每个 w，

$$w \in L \Rightarrow \Pr[M \text{ 拒绝}] \le \epsilon, \qquad w \notin L \Rightarrow \Pr[M \text{ 接受}] \le \epsilon$$

定义：$BPP = \{L \mid \text{某多项式时间 } \mathrm{PTM} \text{ 以误差 } 1/3 \text{ 判定 } L\}$。

（等价地可用任何常数 $< 1/2$。）

20.3 定理与证明思路

定理（放大引理, amplification）：若 M_1 以误差 $\epsilon_1 < 1/2$ 判定 L，则对任意 $\epsilon_2 < 1/2$ 存在同语言的 PTM M_2 以误差 ϵ_2 判定；进一步可做到 $\epsilon_2 < 2^{-n^c}$。

证明：跑 M_1 共 k 次取多数。由 Chernoff 界，错误率随 k 指数下降。**代价是 k 倍时间**，但仍是多项式。

意义：误差可以被压到"比任何多项式倒数都小"，于是"随机算法出错"在实践中可忽略。这是 BPP 成为"现实可解"合理模型的原因。

定理（$EQ_{ROBP} \in BPP$）：读一次分支程序（read-once branching program, ROBP）的等价性问题。

证明思路（算术化 + 多项式恒等测试）：

1. **算术化（arithmetization）**：把布尔运算换成算术运算：$\phi \wedge \psi \mapsto \phi\psi$，$\bar{\phi} \mapsto 1 - \phi$，$\phi \vee \psi \mapsto \phi + \psi - \phi\psi$。由于 ROBP 无环且每个变量只读一次，执行路径只经过每个节点一次，这套规则给出的多项式**精确等于** BP 的布尔函数在 $\{0,1\}^n$ 上的取值。
2. 于是 $B_1 \equiv B_2 \iff p_1 \equiv p_2$（作为多项式）。
3. 若 $p_1 \neq p_2$，则 $p = p_1 - p_2 \neq 0$，由 **Schwartz–Zippel 定理**，在有限域 $\mathbb{F}_q$ 上随机取点 r 有

$$\Pr[p(r) = 0] \le \frac{n \cdot d}{q}$$

其中 d 是每个变量的次数上界（ROBP 下 $d = 1$），n 是变量数。取 q 足够大（如 $q > 2n$），错误率 $\le 1/3$。

1. 随机取点赋值、求值两个多项式、比较结果——多项式时间。

Schwartz–Zippel 定理（L24 原文）：若 $f(x_1, \ldots, x_n) \not\equiv 0$ 且每个变量的次数 $\leq d$，则从 $\mathbb{F}_q$ 随机取点使 $f = 0$ 的概率 $\leq nd/q$。它由一元情形"次数 d 的非零多项式至多 d 个根"归纳而来。

为什么必须用"非布尔输入"：若只在 $\{0,1\}^n$ 上比较，两个不同的多项式可能只在极少点上不同（如只在 1 个赋值上不同），采样需要指数次。算术化允许在**整个域**上取值，才使随机测试有效。L23 的原话是 "Try a different idea: Run B_1 and B_2 on non–Boolean inputs."

20.4 例子

例（EQ_{BP} 是 coNP 完全的）：一般分支程序（可重复读变量）的等价性是 coNP 完全的（hw6 第 5 题，教材 10.21）。对比之下 $EQ_{ROBP} \in BPP$：**"读一次"这个限制把问题从 coNP 完全降到 BPP**。这个对比是本章最重要的结构性事实。

20.5 常见误区

误区	为什么错	正确做法
认为 BPP 算法"一定对"	是"大概率对"，误差可放大但非零	说"以概率 $1 - 2^{-n^c}$ 正确"
把放大当成免费	需要重复 k 次，时间乘 k	明确写出时间开销
只用布尔赋值做多项式测试	差异可能极小，采样不够	在有限域上取随机点

第 21 章 交互式证明与 IP

> **对应材料**：Lec 25–26（Sipser §10.4）｜关键词：interactive proof, IP, graph isomorphism, #SAT, coNP ⊆ IP

21.1 问题：证明一定要"写下来"吗

NP 的证书是**静态**的：一段可被多项式时间验证的字符串。交互式证明（interactive proof）允许**动态**证明：验证者掷硬币、提问，证明者回答，多轮交互后验证者被说服。

这能不能扩大可证明的范围？答案是惊人的：**能扩到 PSPACE**。

21.2 形式定义

两方：验证者 V（概率多项式时间 TM）、证明者 P（计算能力无限制）。双方看到输入 w，交换多项式条多项式长的消息，最后 V 接受或拒绝。

$$IP = \{L \mid \exists V\ \exists P:\ w \in L \Rightarrow \Pr[(V \leftrightarrow P) \text{ 接受}] \geq 2/3,\ \ w \notin L \Rightarrow \forall P^*\ \Pr[(V \leftrightarrow P^*) \text{ 接受}] \leq 1/3\}$$

注意量词结构："**存在诚实的 P" 对 "任意作弊的 P^***"。前者对应完全性（completeness），后者对应可靠性（soundness）。这个量词形状与 NP 的定义（∃ 证书）很像，但 P 是**交互的**、V 是**随机的**。

21.3 定理与证明思路

定理（$GI \in IP$）：图同构问题

$$GI = \{\langle G, H\rangle \mid G \cong H\}$$

的**补**（非同构）在 IP 里。协议（L25）：

1. V 随机选 G 或 H 之一，随机置换节点得到 K，把 K 发给 P；
2. P 回答"K 来自 G"或"来自 H"；
3. 重复两轮，两次都对则接受。

正确性：若 $G \ncong H$，P 能准确判断（因为 K 只可能同构于其中一个），故必对；若 $G \cong H$，则 K 同构于两者，P 只能瞎猜，两轮全对概率 $1/4$。

为什么这很重要：GI 既不知道是否在 P、也不知道是否 NP 完全、甚至不知道是否在 $coNP$——但它**在 IP 里**。这说明 IP 包含了 NP 之外的东西。

定理（$\#SAT$ 是 coNP 难的，且 $\#SAT \in IP$）：

$$\#SAT = \{\langle \phi, k\rangle \mid \phi \text{ 恰有 } k \text{ 个可满足赋值}\}$$

- coNP 难：SAT 的补可以归约到它（$k = 0$）；
- $\in IP$：L25–L26 的主证明。

协议的核心思想（"逐变量求和 + 随机挑战"）：设 ϕ 有 n 个变量，定义

$$g(x_1, \ldots, x_n) = \text{算术化后的 } \phi, \qquad \#\phi = \sum_{x_1, \ldots, x_n \in \{0,1\}} g(x_1, \ldots, x_n)$$

把求和按变量逐层剥开：定义 $g_i(b_1, \ldots, b_i) = \sum_{x_{i+1}, \ldots, x_n} g(b_1, \ldots, b_i, x_{i+1}, \ldots, x_n)$。$P$ 声称 $\#\phi = k$，即 $g_0 = k$。

协议（L26 的 version 1）：

轮	P 发送	V 检查
0	g_0 作为多项式	$g_0() = k$
1	$g_1(x)$ 作为多项式	$g_1(0) + g_1(1) = g_0$，然后随机取 r_1
2	$g_2(r_1, x)$	$g_2(r_1, 0) + g_2(r_1, 1) = g_1(r_1)$，随机取 r_2
…	…	…

轮	发送	检查
$n+1$	—	$g_n(r_1,\dots,r_n) = g(r_1,\dots,r_n)$

为什么有效：P 若撒谎，第一步的 g_1 就与实际不符，于是 $g_1(0)+g_1(1) \neq g_0$ 会被抓；若 P 在某个多项式上撒谎但恰好通过求和检查，则由 Schwartz–Zippel，随机点 r_i 恰好落在"差异点"上的概率 $\leq d/q$，n 轮累计仍很小。

技术细节（L26 指出）：g 的次数在每个变量上 $\leq |\phi|$，所以域要取足够大（$q > 3|\phi|$ 之类），误差才能压到 $1/3$ 以下。这就是"有限域 $\mathbb{F}_q$"出现的原因。

定理（$coNP \subseteq IP$）：由 $\#SAT \in IP$ 且 $\#SAT$ 是 coNP 难，得每个 coNP 语言都在 IP 里。

定理（$IP = PSPACE$，Shamir 1992）：课程**只陈述不证明**。L25 列出三条事实：(a) $NP \subseteq IP$（V 确定性）；(b) $BPP \subseteq IP$（V 忽略 P）；(c) $IP \subseteq PSPACE$（枚举所有交互，多项式空间）。课上只证较弱的 $coNP \subseteq IP$。

21.4 例子

例（为什么 $IP \subseteq PSPACE$ 是显然的）：V 是多项式时间，消息多项式长，所以整棵交互树是多项式深度、指数宽度。用 DFS 遍历它、在每个叶子上判断接受概率是否 $\geq 2/3$，只需存一条根到叶的路径——多项式空间。**这条观察说明 $IP \subseteq PSPACE$ 不需要任何新思想**，真正的惊奇是反方向。

21.5 常见误区

误区	为什么错	正确做法
把 IP 的可靠性写成"存在 P^* 使…"	必须是**任意** P^* 都无法欺骗	量词顺序是定义的核心
认为 V 的计算能力无限制	V 是概率多项式时间	只有 P 无限制
把 $GI \in IP$ 误记为"$GI \in NP$"	GI 的补在 IP 里；GI 是否在 $coNP$ 未知	区分 GI 与 $\overline{GI}$

第 22 章 专题 A：自动机——判定流程与闭包

> 第 1–3 章的压缩版。**完整的闭包表、泵引理公式、决策树在配套文件《18404_计算理论_项目上手速查.md》里**，本章只留"命题如何被判定"的骨架，避免两份文档重复。

22.1 给一个语言，按顺序问六个问题

① L 有限吗？ ———————— 是 → 正则
② 能写出正则表达式吗？ ——— 是 → 正则
③ 是"倒数第 k 个符号"型模式？ → 是 → 正则（NFA 反向猜）
④ 由正则语言闭包运算组合而成？ → 是 → 正则
⑤ 与某个正则 R 相交后得到已知非正则语言？ → 是 → 非正则
⑥ 以上都不行 → 上泵引理（选 s，证所有切分都矛盾）

顺序不能颠倒：**先闭包化简，再泵引理**。L3 的"0 与 1 个数相等"就是先交 0^*1^* 再引用已知结论。

22.2 三个必须记住的"为什么"

命题	为什么成立	常见错法
NFA 与 DFA 等价	子集构造，2^k 个状态	以为 NFA "更强"
正则对补封闭	需先确定化，再取 $Q \setminus F$	直接对 NFA 取补
泵引理只能否证	它是必要条件	用它证"正则"

22.3 一个课外补丁

标注：课外。 判断两台 DFA 是否等价有两条路：第 9 章的"对称差 + 空性"只回答是否相等；**最小化**（反复标记可区分状态对，直到稳定）给出规范形，从而也能回答"最少要几个状态"。后者 Sipser 未在课上讲，但在编译器与模型检验里更常用。

第 23 章 专题 B：文法——三个判据的分工

> 第 4–6 章的压缩版。细节表格见速查文件；本章只讲"哪个工具解决哪类问题"。

23.1 三个判据

问题	工具	代价
$w \in L(G)$?	转 CNF 后枚举 $2\|w\| - 1$ 步推导	指数（朴素）/ 多项式（DP，第 15 章）
$L(G) = \varnothing$?	从终结符反向标记变量	多项式
某语言是 CFL 吗?	泵引理（否证）+ 闭包（化简）	—

23.2 CNF 转换的四件事

1. 加 $S_0 \to S$，让起始变量不出现在任何规则右部；
2. 消 ε 规则（起始变量除外），注意可空变量的连锁；
3. 消单位规则 $A \to B$（用传递闭包展开）；
4. 拆长规则与混合规则（$A \to B_1 \cdots B_k$ 二分；$A \to aB$ 把 a 提成单独变量）。

为什么必须做第 2、3 步：A_{CFG} 的判定器依赖"推导长度恰为 $2|w|-1$"，而这个引理的证明前提是"每条规则要么二元要么产出一个终结符"。少做一步，长度界就不成立。

23.3 一条不能推广的推论

第 5 章的推论是 $\text{CFL} \cap \text{regular} = \text{CFL}$。它**不能**改成 $\text{CFL} \cap \text{CFL} = \text{CFL}$——后者为假（hw2 第 1 题）。两个命题只差一个词，是本课最经典的"背错一句就失分"的点。

第 24 章 专题 C：可计算性——归约方向与三个模板

> 第 9–13 章的压缩版。归约图谱与判定性全景表在速查文件里；本章只讲**写作模板**与**为什么课程禁用 Rice 定理**。

24.1 三个模板

模板 1（证不可判定）：假设 R 判定靶子 B → 构造 S 判定 A_{TM} → S = "on $\langle M, w\rangle$：造辅助对象 → 用 R 询问 → 按答案翻转" → 矛盾。

模板 2（证不可识别）：给出可计算 f 使 $\langle M, w\rangle \in A_{\text{TM}} \iff f(\langle M, w\rangle) \in B$；若 B 可识别则 A_{TM} 可识别，矛盾。

模板 3（证可判定）：给判定器 + 一个"有限性"论证：步数有限、格局数有限、或推导长度有限。

24.2 方向：唯一的失分大项

$A \le_m B$ 的含义是"A 不比 B 难"。因此：

- 要证 B **不可判定**，必须让**已知不可判定的** A_{TM} 在左边；
- 若写成 $B \le_m A_{\text{TM}}$，只说明 B 不太难，什么也推不出。

L9 专门讨论了 "reduce" 一词的两种读法，并指出记号 $\le$ 支持"把 A 的难度降到 B"。写作时把这句话写在证明开头，能避免 90% 的方向错误。

24.3 为什么课程禁用 Rice 定理

标注：课外补充。 Rice 定理说：任何非平凡的语言性质都不可判定，一次覆盖 E_{TM}、ALL_{TM}、正则性检测等一大类。样题（Fall 2001 期中第 3 题）明确写 "Do not use Rice's theorem"，原因是它**跳过了本课要训练的核心技能**——亲手构造归约。考试里用它即使结论对，也拿不到过程分。

24.4 硬度阶梯

A_{TM}（可识别、不可判定）→ E_{TM}（不可识别）→ EQ_{TM}（连补都不可识别）。这条阶梯的意义：**"不可判定"不是二值的**，不同问题在"能否被部分验证"上有真实差别。

第 25 章 专题 D：复杂性——类关系与电路复杂性概览

> 第 14–21 章的压缩版。完整类关系图在速查文件；本章补两块正文没展开的东西：**已知与未知的分界**、**电路复杂性为什么不在本课**。

25.1 已知与未知的分界

关系	状态	依据
$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq EXPTIME \subseteq EXPSPACE$	已知包含	定义与模拟
$PSPACE = NPSPACE$	已知相等	Savitch
$NL = coNL$	已知相等	Immerman–Szelepcsényi
$NL \subsetneq PSPACE$	已知严格	空间层次定理
$P \subsetneq EXPTIME$	已知严格	时间层次定理
$P = NP$、$L = NL$、$NP = coNP$、$P = PSPACE$	**开放**	—

一条容易被记错的推论：层次定理给出的是 $NL \subseteq SPACE(\log^2 n) \subsetneq PSPACE$，即"$NL$ 与 $PSPACE$ 分离"，**不是** "$L \neq NL$"。

25.2 完全性的归约必须比类更弱

类	完全性用的归约	典型完全问题
NP	$\leq_P$	SAT、$3SAT$、$CLIQUE$、$HAMPATH$
$PSPACE$	$\leq_P$（$\leq_{\log}$ 也可）	$TQBF$、GG

类	完全性用的归约	典型完全问题
NL	$\leq_{\log}$	$PATH$、$2SAT$
$EXPSPACE$	$\leq_P$	$EQ_{REX\uparrow}$

若归约强于类本身，类内所有问题会互相归约、全部变成完全问题，概念就废了（第 18 章 check-in 18.1 的原话逻辑）。

25.3 电路复杂性概览（课外）

标注：18.404 的 Readings 范围停在 §10.4，不含 Sipser §9.3 电路复杂性。以下是概览，供与 18.405 衔接。

布尔电路是由 AND/OR/NOT 门构成的有向无环图，n 个输入、1 个输出；**电路族** $\{C_n\}$ 对每个输入长度给一个电路。

关键差别是**非一致性（non-uniformity）**：TM 是"一台机器管所有长度"，电路族允许每个 n 用完全不同的电路，于是可以"免费"携带建议串（advice）。后果：

概念	含义
$SIZE(s(n))$	电路大小 $\leq s(n)$ 可计算的函数类
$P/poly$	多项式大小电路族；**包含不可判定语言**（advice 任意）
$NP \subseteq P/poly$?	若成立则多项式层次坍塌（Karp–Lipton），故一般认为不成立

为什么本课不讲：电路下界需要额外的组合工具（随机限制法、多项式方法），且与"一致性模型"主线不同。它通常出现在 18.405（Advanced Complexity）——L26 最后一页正好推荐了这门课。

25.4 一句话总结

复杂度理论的张力在一句话里：**给更多资源能不能多算？层次定理说"能"，但只对差距足够大的类；对 P 与 NP 这种只差一个指数符号的相邻类，我们至今只会说"不知道"。**

第 26 章 与已有笔记的衔接

> 本目录已有两份与本课强相关的笔记，本章说明"哪些内容不必重读、哪些必须重读"。

26.1 ← CS70：证明的地基

`UCB_CS70_离散数学与概率论_学习笔记.md` 覆盖了本课全部证明技术的前置：直接证明、逆否、反证、归纳、强归纳、鸽笼原理、图论基础、条件概率与期望。

本课**直接复用**的三处：鸽笼原理（第 3 章泵引理）、归纳（第 19 章层次定理的构造）、概率放大（第 20 章 Chernoff / 多数投票）。**需要新建**的能力只有两样：**把"计算"形式化成对象**（第 7 章），以及**用对角线构造反例**（第 10 章）——这两样 CS70 没有。

26.2 ← CS170：归约的工程版

`UCB_CS170_算法设计_学习笔记.md` 用了 5–6 次课讲 NP 完全性，重心是 gadget 设计与近似算法。两边的分工已在 §0.7 给表，这里只补三条操作建议：

1. **CS170 的 2-SAT ↔ 本课的 2SAT**：同一个问题，一个给线性算法，一个证 NL 完全。对照着看，能同时理解"算法"与"类的定位"两种视角。
2. **CS170 的归约靶子（3-SAT、Independent Set）在本课只出现一次**（第 16 章 $3SAT \leq_P CLIQUE$）。本课真正的靶子是 A_{TM} 与 $TQBF$——**不可判定性与 PSPACE 完全性在 CS170 里没有对应物**。
3. **不要在本课笔记里重抄近似比**。18.404 一次都没讲近似算法，抄进来只会稀释主线。

26.3 ← 6.006 与 18.405

`MIT_6006_算法导论_学习笔记.md` 的图算法部分（BFS/DFS/最短路）在本课只作为 $PATH \in P$ 的背景出现；本课对它的唯一要求是"能写出 $PATH$ 的多项式判定器并核算复杂度"（第 14 章）。

若想继续深入，L26 最后一页推荐了三门课：18.405/6.841（Advanced Complexity，2021 秋）、18.425/6.875（Cryptography，2021 秋）、6.842（Randomness and Computation）。本笔记第 25 章的电路复杂性概览就是 18.405 的入口。

第 27 章 复习清单与自测题

27.1 复习清单（按"必须能当场写出"分级）

级别	内容	出自
A（必写）	DFA 五元组、NFA 五元组、TM 七元组、CFG 四元组、PDA 六元组	第 1、2、4、5、7 章
A	子集构造的四个分量（含 $F' = \{R \mid R \cap F \neq \varnothing\}$）	第 2 章
A	正则泵引理三条件 + 0^n1^n 的否证	第 3 章
A	CFL 泵引理三条件 + $0^n1^n2^n$ 的否证	第 6 章

级别	内容	出自
A	A_{TM} 不可判定的对角线证明（含 D 的构造）	第 10 章
A	归约模板（证不可判定 / 证不可识别）与**方向**	第 11 章
A	Cook–Levin 的四个子公式名与作用	第 16 章
B（能复述）	GNFA → RE 的归纳式 $R_{ij} \cup R_{i,\mathrm{rip}}(R_{\mathrm{rip},\mathrm{rip}})^* R_{\mathrm{rip},j}$	第 3 章
B	A_{CFG} 判定器为什么需要 CNF	第 9 章
B	Savitch 的分治 + 空间 $O(f^2)$	第 17 章
B	$TQBF$ 构造中 ∀ 如何把"复制"变"共享"	第 17 章
B	$NL = coNL$ 的"数数 + 校验"	第 18 章
B	层次定理的 $o(f/\log f)$ 与三个技术修正	第 19 章
C（知道结论）	$EQ_{REX\uparrow}$ 是 EXPSPACE 完全、$IP = PSPACE$、$EQ_{ROBP} \in BPP$	第 19–21 章
C	PCP 不可判定、E_{LBA} 不可判定	第 12 章

27.2 自测题（10 题，答案紧随）

Q1. 设 $L = \{w \in \{0,1\}^* \mid w$ 的倒数第 3 个符号是 $1\}$。它是正则的吗？给出最小 DFA 的状态数。

Q2. 用泵引理证明 $L = \{0^n 1^{2n} \mid n \geq 0\}$ 不正则。

Q3. 为什么 A_{CFG} 的判定器要先转 CNF？给出不转 CNF 会失败的原因。

Q4. 证明：若 $A \leq_m B$ 且 B 可识别，则 A 可识别。

Q5. E_{TM} 与 ALL_{TM} 都可判定吗？分别说明。

Q6. 为什么 EQ_{DFA} 可判定而 EQ_{CFG} 不可判定？给出一句话原因。

Q7. Cook–Levin 中 ϕ_{move} 为什么用 2×3 窗口而不是整行？

Q8. $PSPACE = NPSPACE$ 与 $P = NP$ 的证明难度差别在哪？

Q9. 设 $L \in NP$，能否断言 $\overline{L} \in NP$？说明理由。

Q10. 为什么"NP 完全问题没有多项式算法"不是定理？

答案

A1. 是正则的。倒数第 3 个符号意味着只需记住"最近 3 个符号"，共 $2^3 = 8$ 个状态（或按 NFA 反向猜：$1\Sigma\Sigma$ 的 NFA 只需 4 个状态）。2^k 下界由 hw1 第 5 题（教材 1.61）给出：任何 DFA 至少 2^k 个状态。

A2. 假设正则，取 p，令 $s = 0^p 1^{2p}$。由 $|xy| \leq p$，$y = 0^k$（$k \geq 1$）。则 $xy^0 z = 0^{p-k} 1^{2p}$ 中 0 与 1 的比例不再是 $1:2$，不在 L，矛盾。

A3. 不转 CNF 时规则可能是 $A \to \varepsilon$ 或 $A \to B$ 或长规则，推导长度没有统一上界，无法"枚举 $2|w| - 1$ 步"。CNF 保证每条规则要么二元要么产出一个终结符，于是任何推导恰有 $2|w| - 1$ 步，枚举才有限。

A4. 设 M_B 识别 B。构造 M_A = "On input w：算 $f(w)$，在 $f(w)$ 上跑 M_B；M_B 接受则接受，M_B 拒绝或不停机则同样。"由 $w \in A \iff f(w) \in B$，M_A 接受 A，故 A 可识别。

A5. 都**不**可判定：E_{TM} 与 ALL_{TM} 都能从 A_{TM} 归约得到。但 E_{TM} 连可识别都不是，而 ALL_{TM} 的补才是可识别的（"存在一个不在语言里的串"）。

A6. DFA 是**识别器**且对交、补封闭，故可用对称差 + 空性判定；CFG 是**生成器**，CFL 对交、补不封闭，同样构造失效。

A7. 因为 TM 一步只改变头所在格及其左右邻格，且只影响相邻两行——局部性保证"每个 2×3 窗口合法"等价于"整张 tableau 合法"，而窗口数只有 $O(n^{2k})$ 个，可写成多项式大小的公式。

A8. Savitch 用"分治 + 只存一个中间格局"就把 NTM 的空间平方化，因为**空间可复用**；而时间不可复用，NTM 的 $2^{O(t)}$ 条分支必须逐个（或并行）走完，至今没有多项式模拟。故前者已证、后者开放。

A9. 不能。NP 是否对补封闭等价于 $NP = coNP$，是开放问题。唯一能确定的是 $L \in NP \Rightarrow \overline{L} \in coNP$，且 $coNP \subseteq PSPACE$。

A10. 因为 $P \neq NP$ 本身未被证明。NP 完全性给出的是"若它有多项式算法则 $P = NP$"，是条件性结论；把它当定理是 18.404 考试里最常见的概念性错误。

第 28 章 参考来源

> 全部链接于 **2026-11-02** 访问。讲义与作业 PDF 来自课程官方打包文件 `18.404j-fall-2020.zip`（53.9 MB，含 25 份讲义 PDF、25 份视频逐字稿、6 份 problem set、3 份 sample exam、1 份 final 解答），文件名形如 `MIT18_404f20_lecN.pdf`。

28.1 课程页面

页面	URL	用途
课程主页	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/	课程定位、教师、级别
Syllabus	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/syllabus/	先修、评分（35/25/15/25）、三大板块周数
Calendar	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/calendar/	26 讲主题与作业截止
Readings	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/readings/	每讲的 Sipser 章节号、教材版本、覆盖范围
Lecture Notes	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/lecture-notes/	25 份讲义 PDF 索引
Assignments	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/assignments/	6 份 problem set（2006 版，题号对应教材第 2 版）

页面	URL	用途
Exams	https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/exams/	3 份 sample exam + 1 份解答；开卷规则

28.2 讲义 PDF（25 份）

文件名模式：`https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/resources/mit18_404f20_lecN/` 对应的静态资源 `MIT18_404f20_lecN.pdf`，$N \in \{1, \ldots, 12, 14, \ldots, 26\}$。其中：

讲次	主题	本笔记对应章
1–3	自动机、NFA、泵引理、GNFA	第 1–3 章
4–8	PDA、CFG、CFL 泵引理、TM 及其变体、判定问题	第 4–10 章
9–13	归约、计算历史、递归定理	第 11–13 章
12–16	时间、NP、NP 完全、Cook–Levin	第 14–16 章
17–22	空间、Savitch、PSPACE 完全、博弈、L/NL、层次定理、oracle	第 17–19 章
23–26	BPP、IP、coNP ⊆ IP	第 20–21 章

28.3 作业与考试

文件	内容
`MIT18_404f20_hw1.pdf` … `hw6.pdf`	6 份 problem set；题号对应教材第 2 版（Assignments 页明确说明）
`MIT18_404f20_mid01.pdf`	Sample Midterm, Fall 2001（含 2WAY-PDA 题）
`MIT18_404f20_mid06.pdf`	Sample Midterm, Fall 2006（含 grid 闭合路径题）
`MIT18_404f20_final.pdf` / `final_soln.pdf`	Sample Final, Fall 2006 及解答

28.4 教材

Sipser, Michael. *Introduction to the Theory of Computation*. 3rd ed. Cengage Learning, 2012. ISBN 9781133187790.（Readings 页指定版本。）

28.5 声明

- 本笔记为个人学习整理，讲义、作业、考卷版权归 MIT 与作者所有；定理陈述与证明思路用自己的语言重写，未整段复制原文。所有"推断""课外""工程补充"标注处**不是**课程原文，引用时请勿归于 MIT 18.404J。
- 两处讲义内部口径差异已在正文标出：Matiyasevich 定理的年份（L6 写 1970、L10 写 1971）、CFL 泵引理中 p 的常数（L5 取 l^{b+1}）。不影响结论，但引用需注意。